

Sequential recommenders

ШАД, рекомендательные системы
2025/26, лекция 10

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

Сегодня обсуждаем кодирование истории пользователя:

1. Трансформеры над историей пользователя для ранжирования
2. Другие трансформеры
3. Трансформеры в Яндексе

На семинаре – разбор статей про трансформеры над историей пользователя в Pinterest.

Трансформеры над историей пользователя для ранжирования

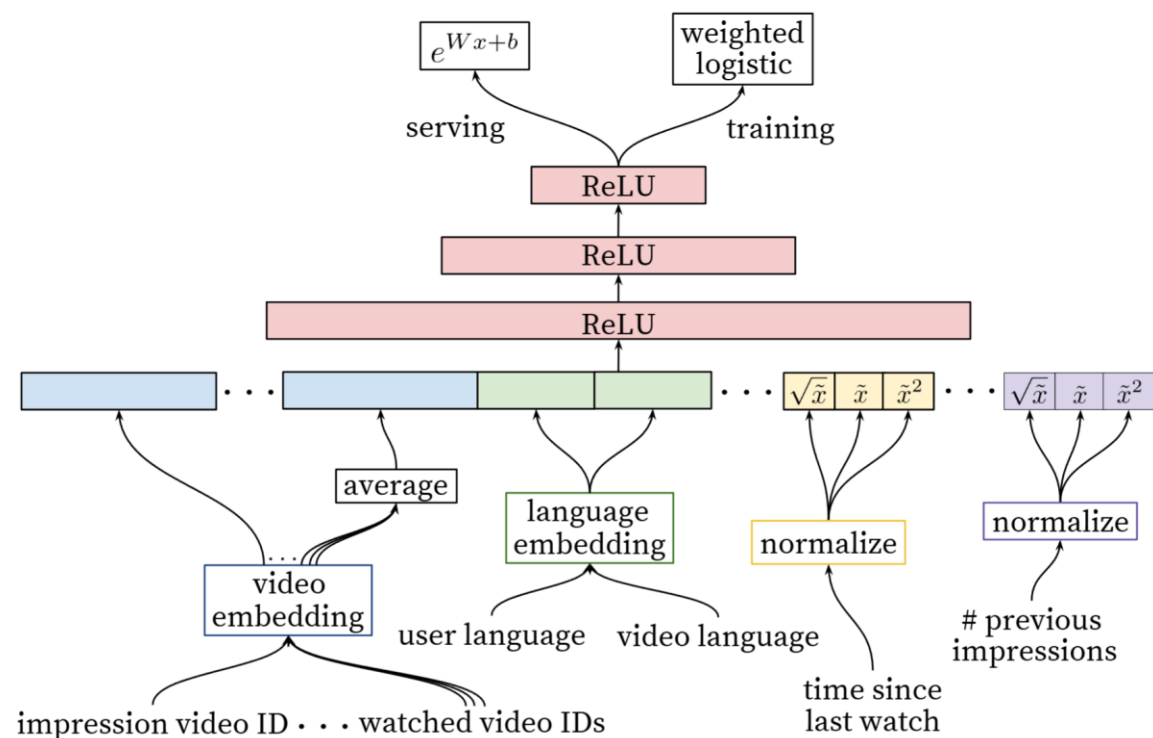
Часть 1

Recap: YouTubeDNN

Векторные представления пользователя в YoutubeDNN:

- Усреднение эмбеддингов последних 50 просмотров пользователя
- Усреднение эмбеддингов униграмм и биграмм последних 50 поисковых запросов пользователя

Формируем вектор пользователя вне зависимости от текущего кандидата.



Deep interest network

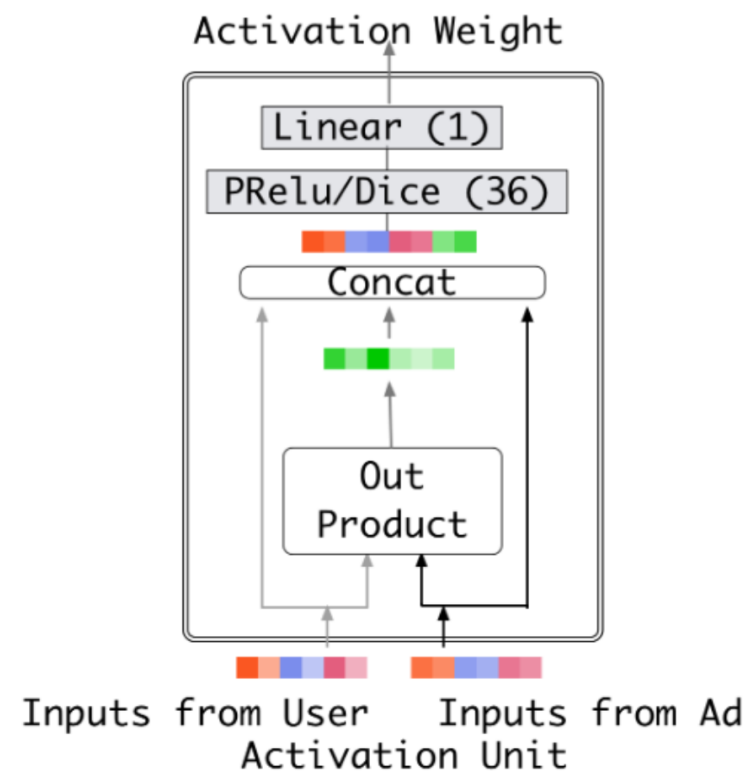
Target-aware attention – attention между историей пользователя и кандидатом

$$\mathbf{v}_U(A) = f(\mathbf{v}_A, \mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_H) = \sum_{j=1}^H a(\mathbf{e}_j, \mathbf{v}_A) \mathbf{e}_j = \sum_{j=1}^H \mathbf{w}_j \mathbf{e}_j,$$

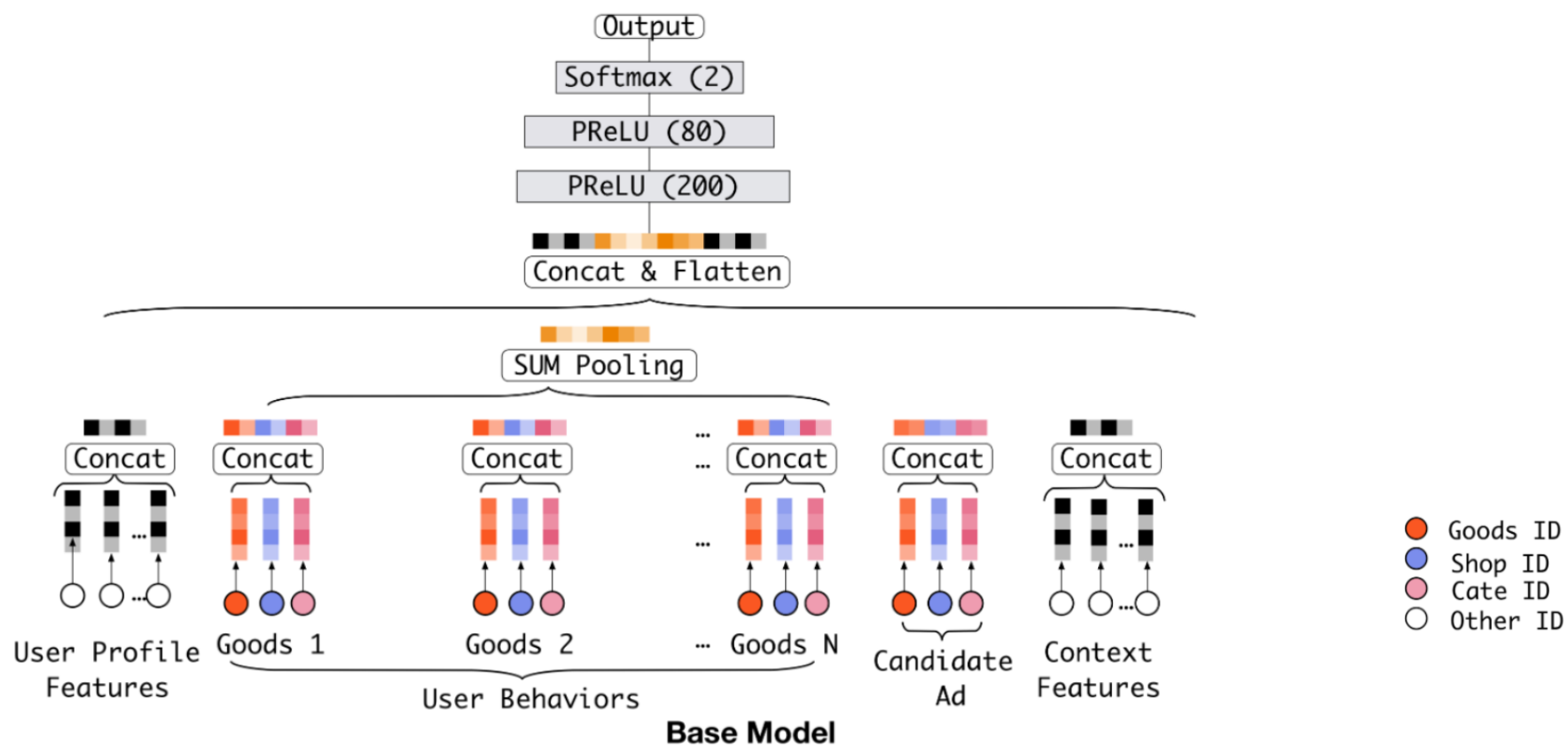
где $\mathbf{e}_1, \dots, \mathbf{e}_H$ - эмбединги исторических событий,
 $\mathbf{v}_A$ - эмбединг кандидата.

Не использовали софтмакс-нормализацию аттеншн скоров – интерпретируют $\sum_{j=1}^H \mathbf{w}_j$ как интенсивность пользовательского интереса.

Это пулинг событий, в котором веса зависят от кандидата.



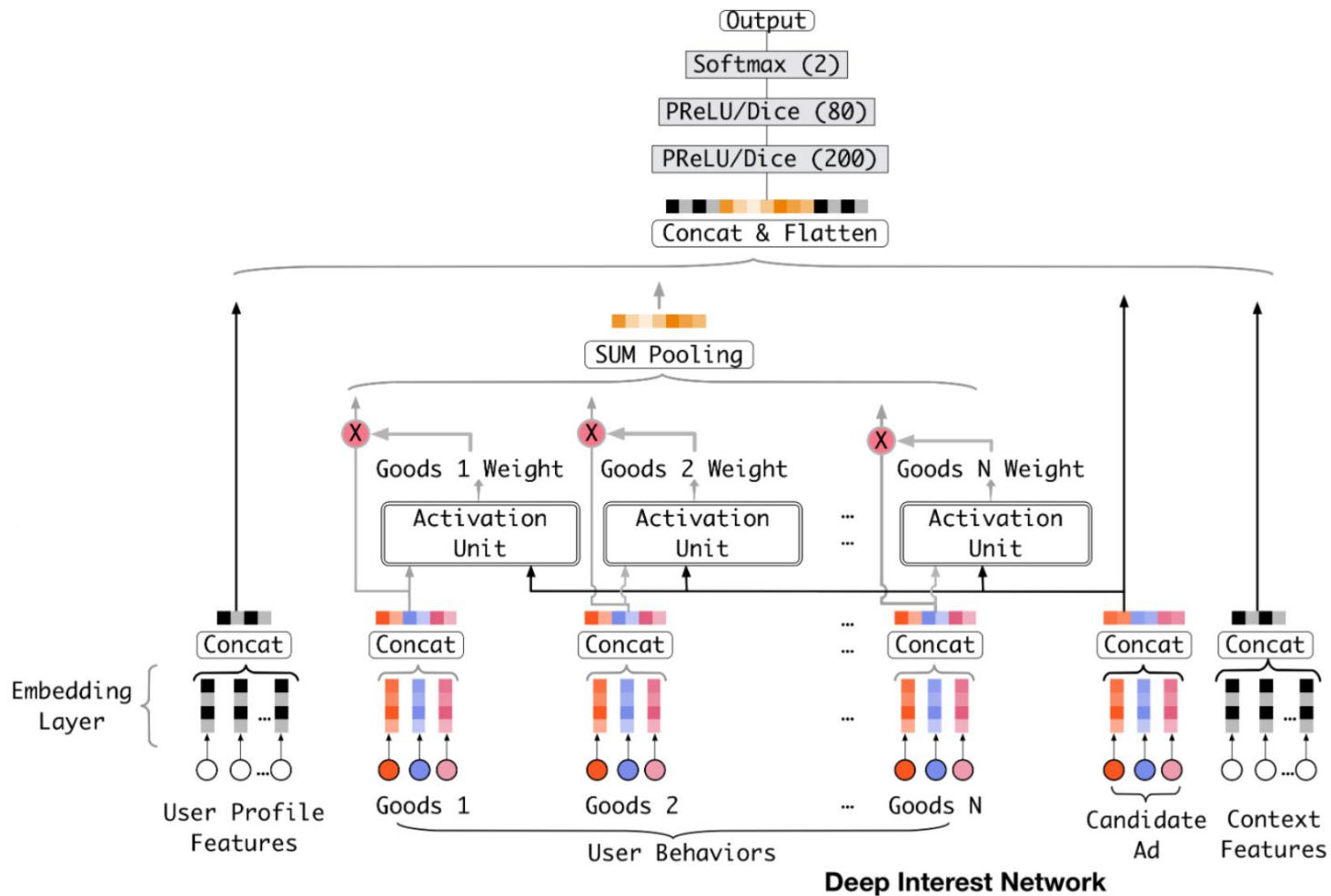
Base model



Базовая модель по схеме YoutubeDNN.

[Deep Interest Network for Click-Through Rate Prediction](#)

Deep interest network



Заменили average pooling на attention.

[Deep Interest Network for Click-Through Rate Prediction](#)

Deep interest network: setup

- Предсказываем вероятность клика на рекламу товара в Alibaba
- История кликов пользователя
 - Вектор события – конкатенация обучаемых эмбеддингов товара, магазина и основной категории товара
- В ранжирующей модели на входе 16 групп признаков, для каждой получаем эмбеддинг размерности 12
- Обучающие данные
 - Две недели – трейн, один день – тест
 - 2 млрд обучающих примеров

Model	AUC	RelaImpr
LR	0.5738	- 23.92%
BaseModel ^{a,b}	0.5970	0.00%
Wide&Deep ^{a,b}	0.5977	0.72%
PNN ^{a,b}	0.5983	1.34%
DeepFM ^{a,b}	0.5993	2.37%
DIN Model^{a,b}	0.6029	6.08%

Deep interest network: results

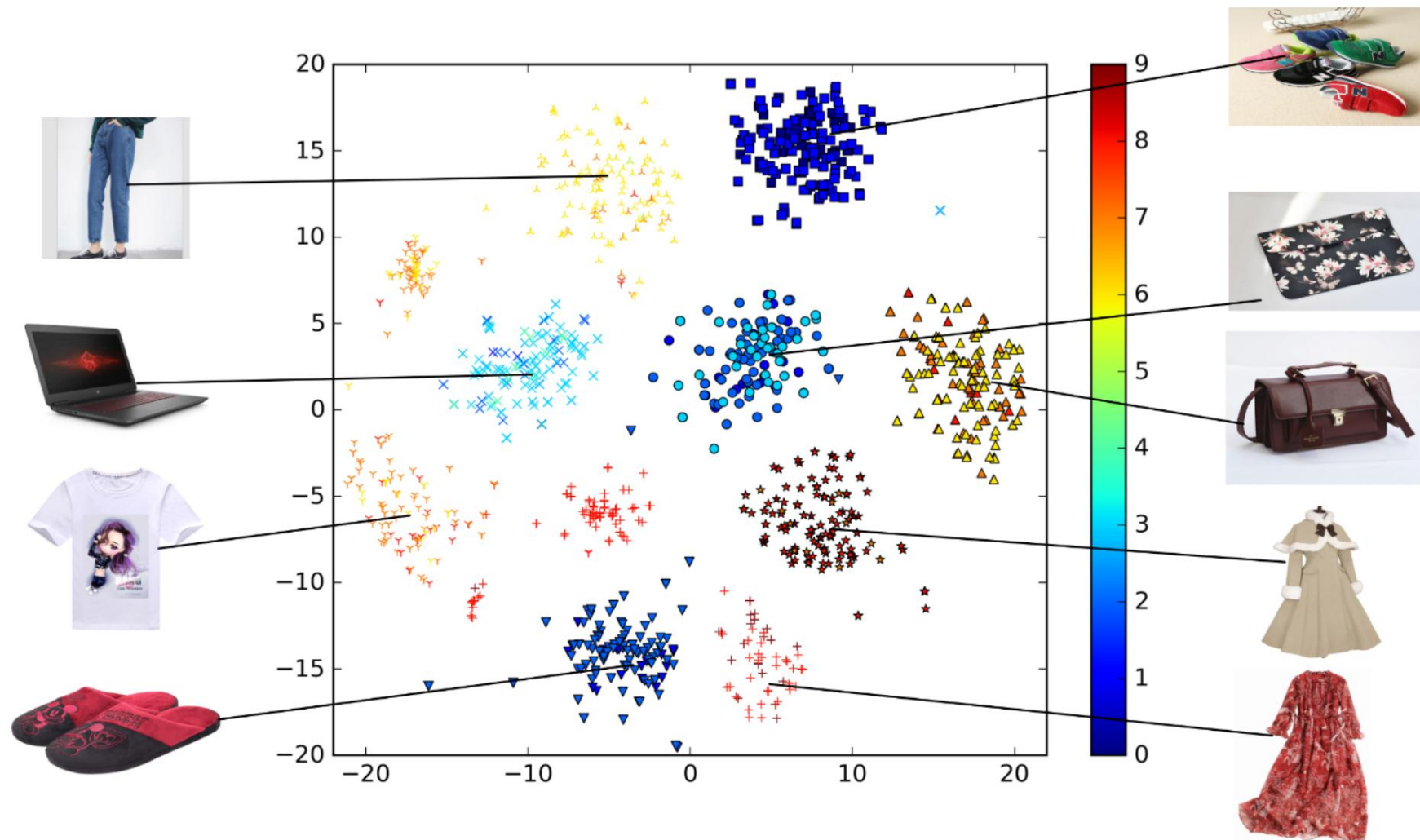
Model	AUC	RelaImpr
LR	0.5738	- 23.92%
BaseModel ^{a,b}	0.5970	0.00%
Wide&Deep ^{a,b}	0.5977	0.72%
PNN ^{a,b}	0.5983	1.34%
DeepFM ^{a,b}	0.5993	2.37%
DIN Model^{a,b}	0.6029	6.08%

Работает лучше, чем базовый YoutubeDNN.

Deep interest network: иллюстрации



Deep interest network: иллюстрации



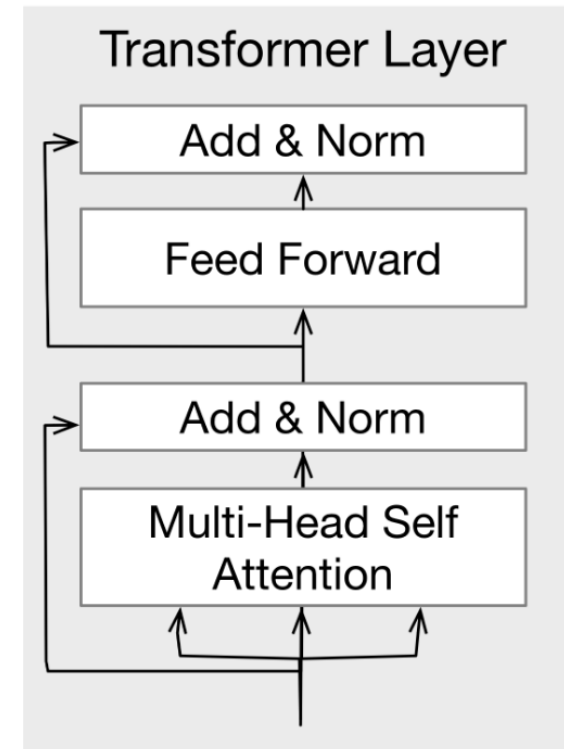
[Deep Interest Network for Click-Through Rate Prediction](#)

Deep Interest Networks

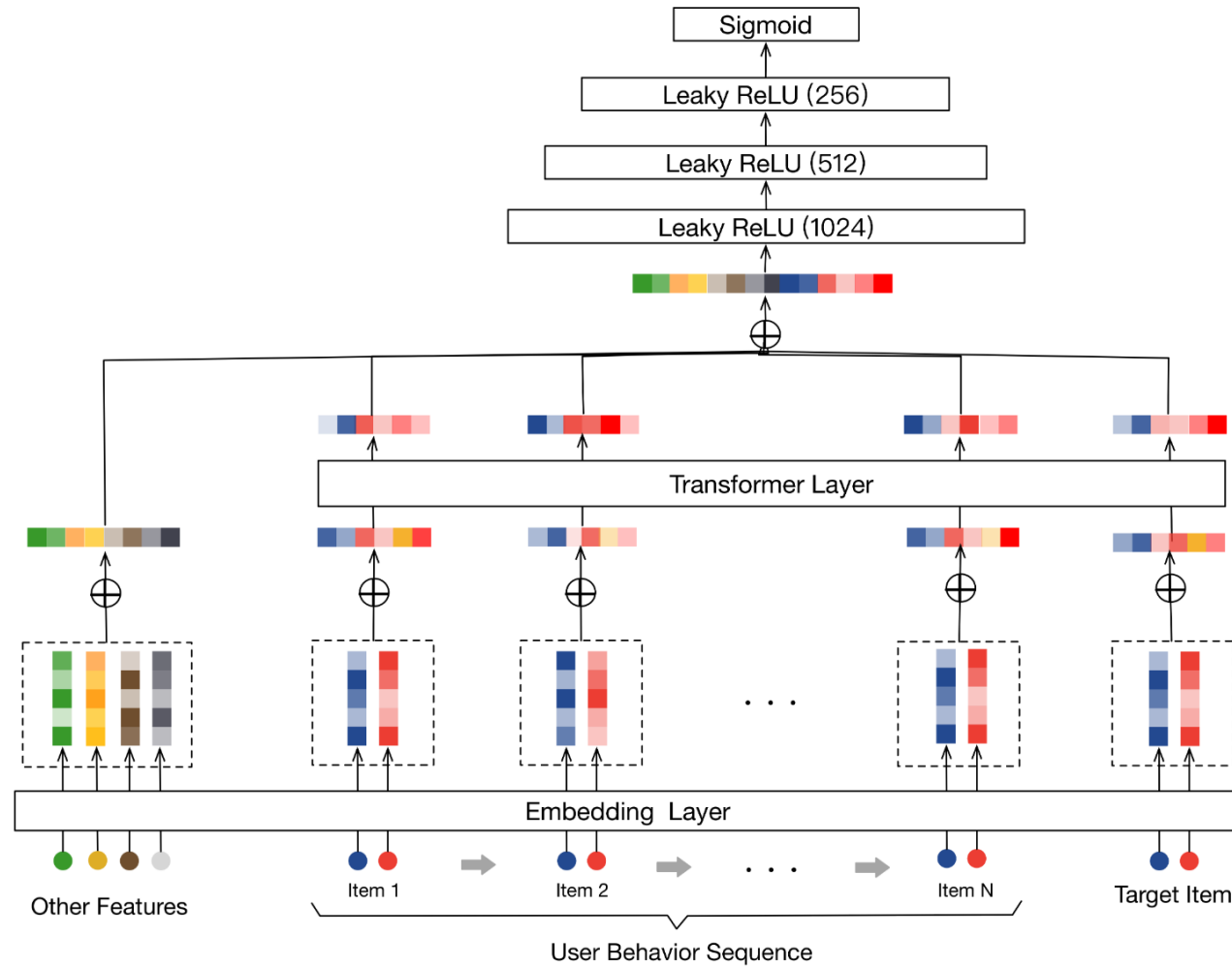
- Deep Interest Evolution Network for CTR Prediction
- Deep Adaptive Interest Network for CTR Prediction
- Deep Context Interest Network for CTR Prediction
- Deep Group Interest Modeling of Full Lifelong User Behaviors for CTR Prediction
- Deep Interest Highlight Network for CTR Prediction in Trigger-Induced Recommendation
- Deep Multi-Interest Network for CTR Prediction
- Deep Evolutional Instant Interest Network for CTR Prediction in Trigger-Induced Recommendations

Behavior sequence transformer

- Хотим учитывать порядок событий
 - Пулинг – не учитывает
- Добавим **позиционные эмбединги** к эмбедингам айтемов при формировании эмбединга события
- Трансформер в качестве энкодера
- Скрытое представление таргет айтема используется как вектор пользователя
 - Конкатится к остальным признакам в ранжирующей модели



Behavior sequence transformer



Behavior sequence transformer

Methods	Offline AUC	Online CTR Gain	Average RT(ms)
WDL	0.7734	-	13
WDL(+Seq)	0.7846	+3.03%	14
DIN	0.7866	+4.55%	16
BST($b = 1$)	0.7894	+7.57%	20
BST($b = 2$)	0.7885	-	-
BST($b = 3$)	0.7823	-	-

BST > DIN.

Pinnability: Pinterest ranking

В Pinterest ранжирующая модель – нейросеть.

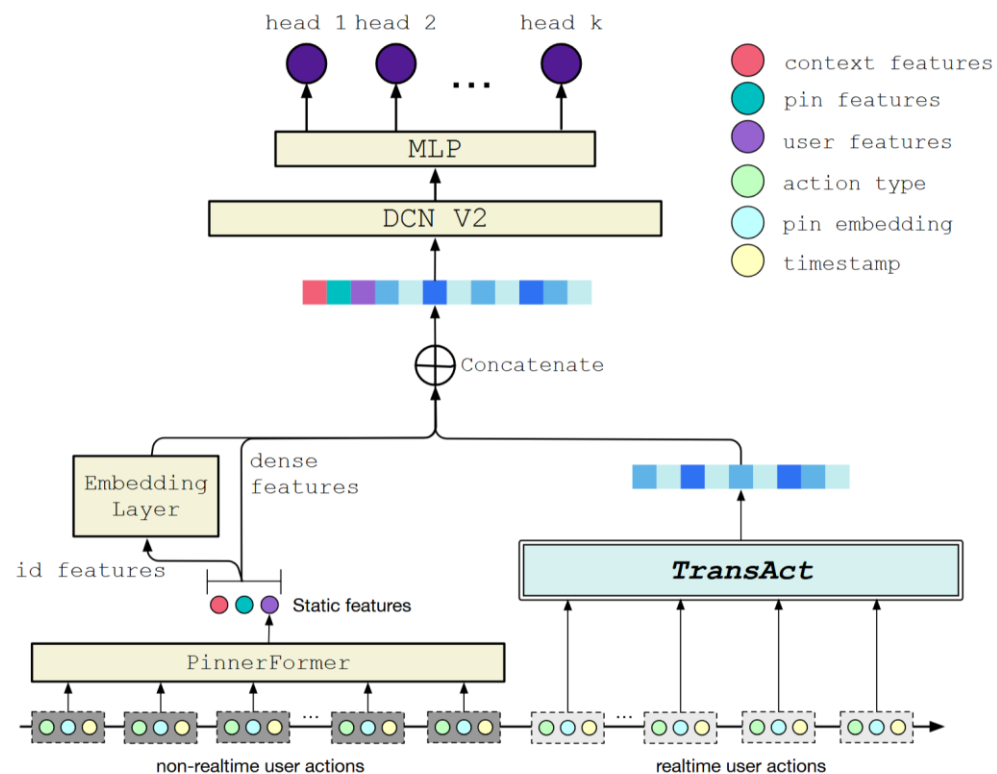
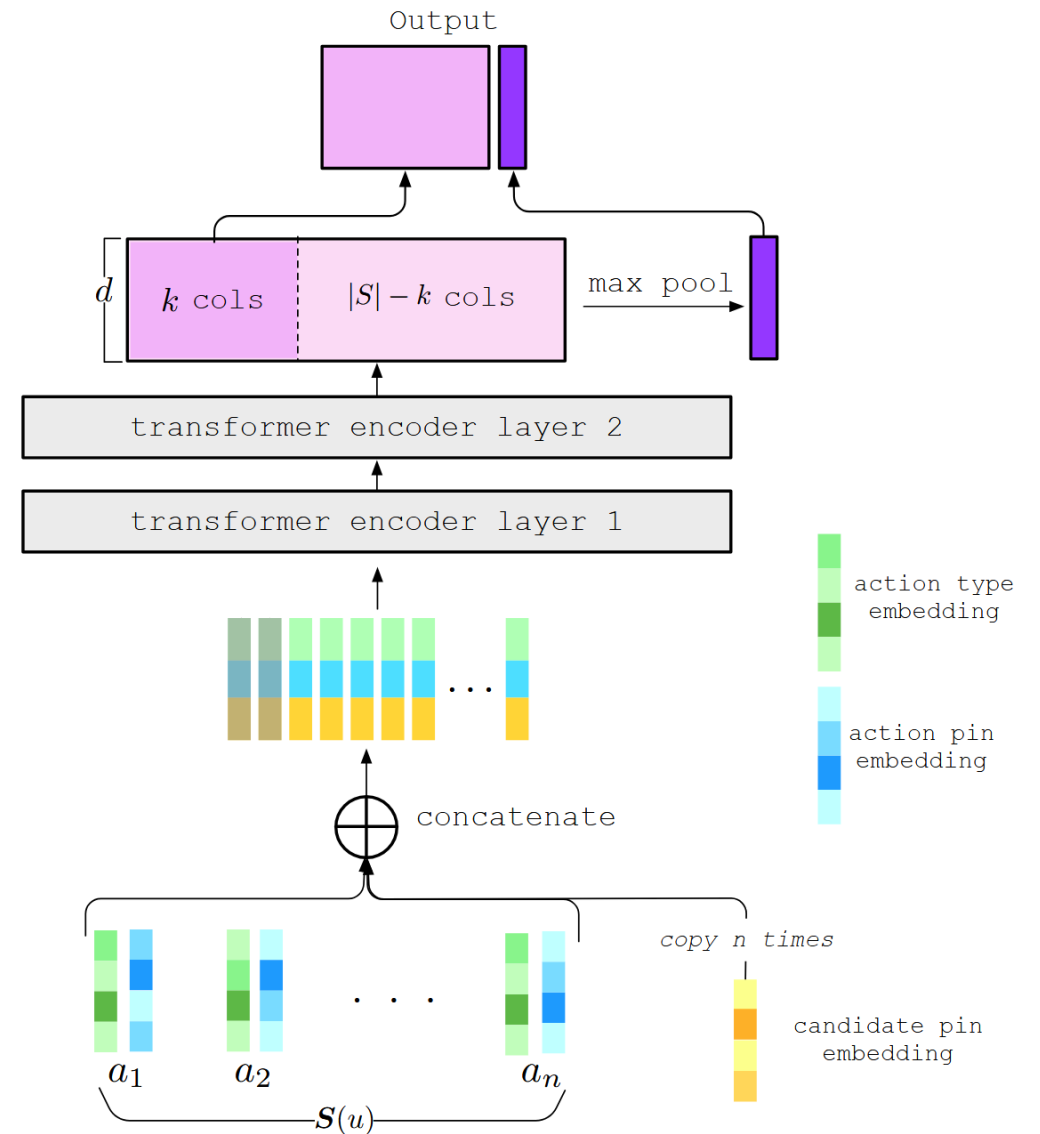


Figure 2: Pinterest Homefeed ranking model (Pinnability)

[TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest](#)

TransAct

- 100 последних действий пользователя в Pinterest
- Есть обучаемый эмбединг типа действия
 - Конкатенируется к вектору айтема
- В качестве эмбединга айтема – PinSage эмбединг
- Двухслойный трансформер
- Конкатенируют вектор кандидата к векторам событий
- Вектор пользователя – конкатенация последних K векторов и макс пулинга



TransAct: results

Table 1: Offline evaluation of comparing existing methods with TransAct. (* statistically insignificant)

Methods	HIT@3/repin		HIT@3/hide	
	all	non-core	all	non-core
WDL + seq	+0.21%	+0.35%	-1.61%	-1.55%
BST (all actions)	+4.41%	+5.09%	+2.33%	+3.59%
BST (positive actions)	+7.34%	+8.16%	-1.12%*	-3.14%*
TransAct	+9.40%	+10.42%	-14.86%	-13.54%

TransAct > BST.

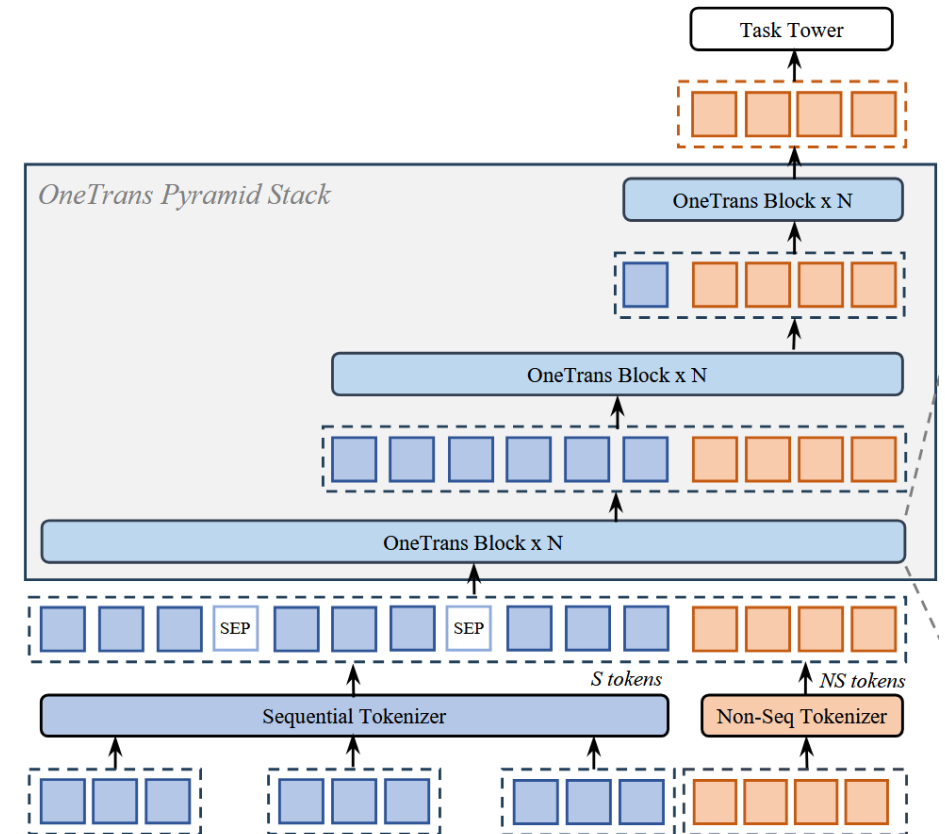
TransAct: results

Table 5: Online evaluation of TransAct

Online Metrics	All Users	Non-core Users
Homefeed Repin Volume	+11.0%	+17.0%
Homefeed Hide Volume	-10.0%	-10.5%
Overall Time Spent	+2.0%	+1.5%

OneTrans

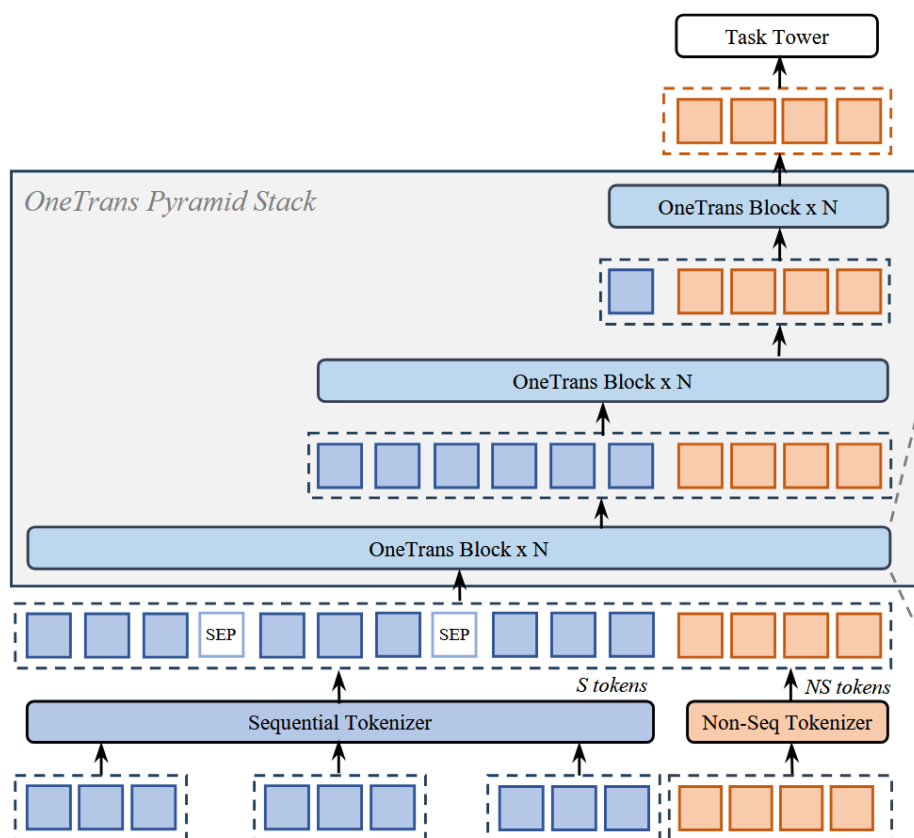
- Рекомендации от ByteDance (вероятно, TikTok)
- Единый трансформер над историей и признаками
- Как в Hiformer – разные token-specific параметры для признаков
 - А для последовательности – одинаковые
- Каузальная маска в трансформере
- Если последовательностей несколько – добавляем SEP token между ними
- Пирамидальные слои



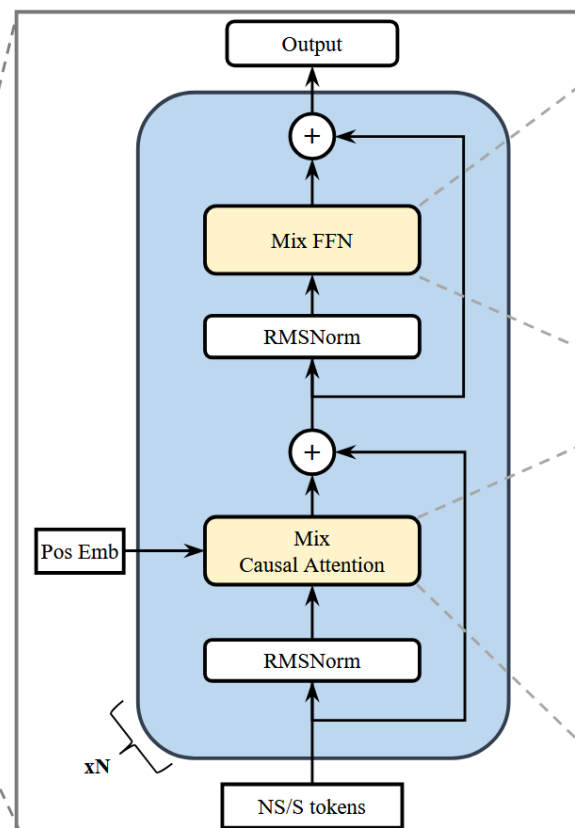
(a) OneTrans Framework

[OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender](#)

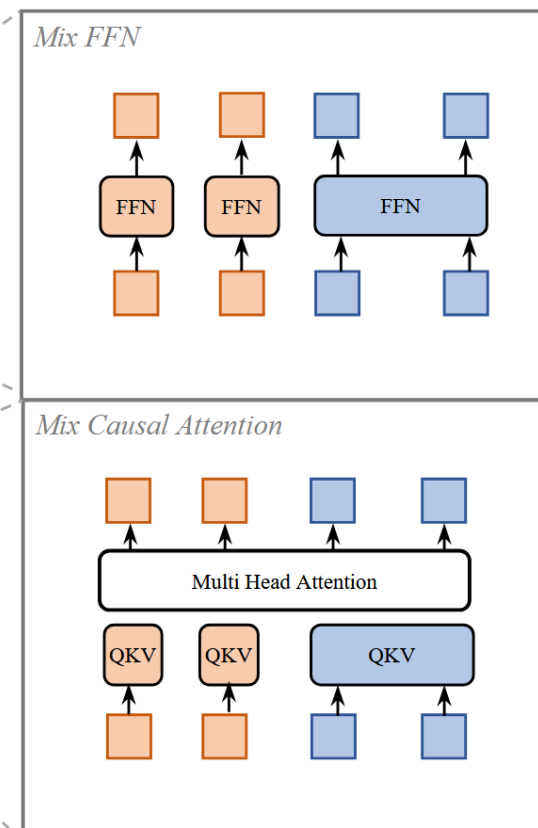
OneTrans



(a) OneTrans Framework



(b) OneTrans Block



(c) Mix Parameterization

[OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender](#)

OneTrans

Table 2: Offline effectiveness (CTR/CVR) and efficiency; higher AUC/UAUC is better. * indicates models deployed in our production in chronological order: *DCNv2+DIN* → *RankMixer+DIN* → *RankMixer+Transformer* → *ONETRANS_S* → *ONETRANS_L*

Type	Model	CTR		CVR (order)		Efficiency	
		AUC ↑	UAUC ↑	AUC ↑	UAUC ↑	Params (M)	TFLOPs
(1) Base model	DCNv2 + DIN (base)*	0.79623	0.71927	0.90361	0.71955	10	0.06
(2) Feature-interaction	Wukong + DIN	+0.08%	+0.11%	+0.14%	+0.11%	28	0.54
	HiFormer + DIN	+0.11%	+0.18%	+0.23%	-0.20%	108	1.35
	RankMixer + DIN*	+0.27%	+0.36%	+0.43%	+0.19%	107	1.31
(3) Sequence-modeling	RankMixer + StackDIN	+0.40%	+0.37%	+0.63%	-1.28%	108	1.43
	RankMixer + LONGER	+0.49%	+0.59%	+0.47%	+0.44%	109	1.87
	RankMixer + Transformer*	+0.57%	+0.90%	+0.52%	+0.75%	109	2.51
(4) Unified framework	ONETRANS _S *	+1.13%	+1.77%	+0.90%	+1.66%	91	2.64
	ONETRANS _L (default)*	+1.53%	+2.79%	+1.14%	+3.23%	330	8.62

OneTrans работает лучше, чем отдельный трансформер над историей и RankMixer.

[OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender](#)

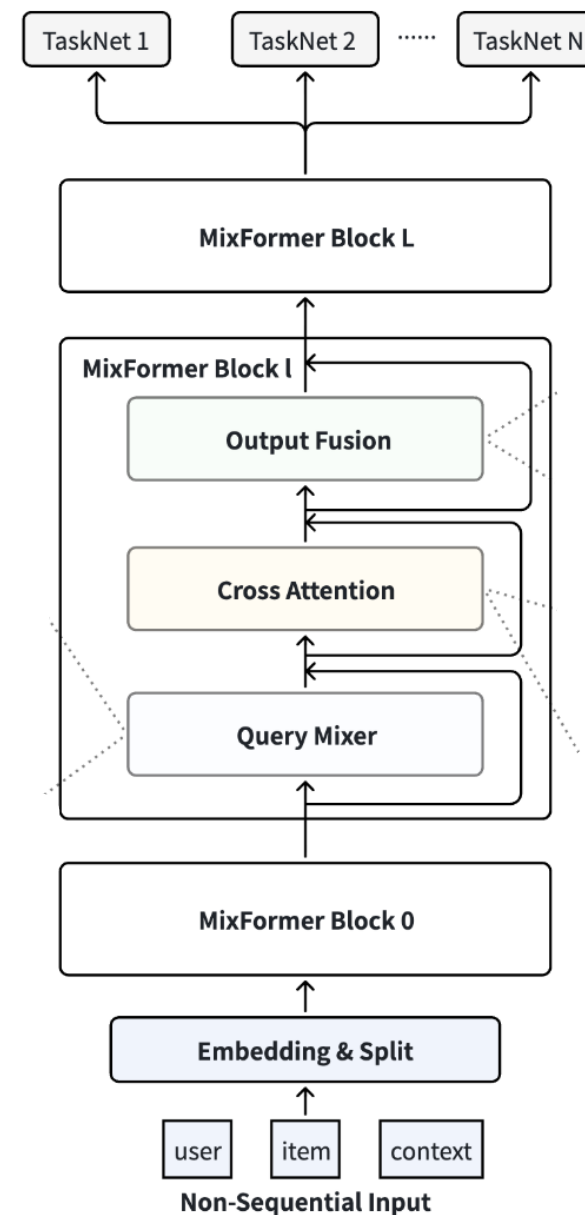
OneTrans

Table 6: Online A/B results: ONETRANS_L (treatment) vs. RankMixer+Transformer (control). Click/u, Order/u, GMV/u, are relative deltas (%). Latency is the *relative* end-to-end per-impression change $\Delta\%$ (lower is better). * denotes $p < 0.05$, and ** for $p < 0.01$

Scenario	click/u	order/u	gmv/u	Latency ($p99$) $\downarrow$
Feeds	+7.737%**	+4.351%*	+5.685%*	−3.91%
Mall	+5.143%**	+2.577%**	+3.670%*	−3.26%

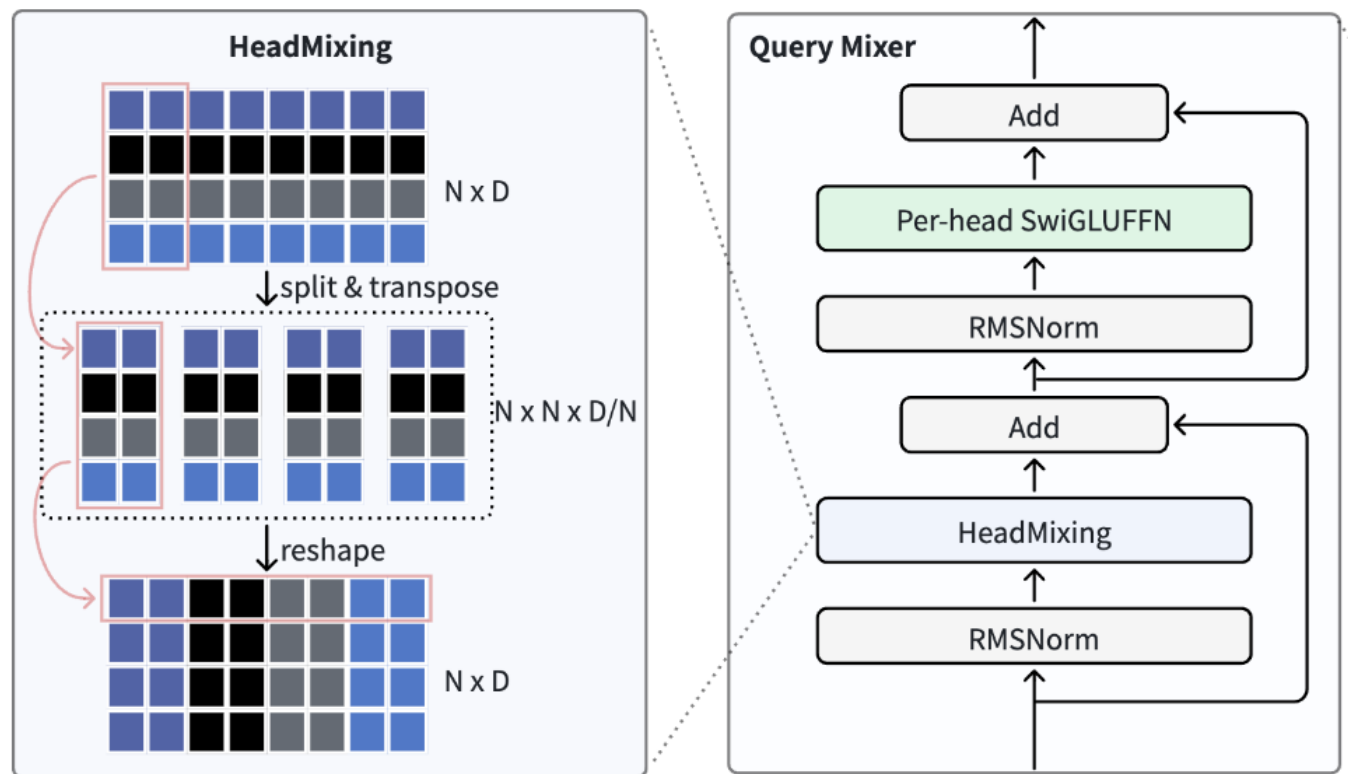
MixFormer

- OneTrans плохо масштабируется
 - Очень дорогой вычислительно
 - Параметры для sequential и для dense частей – отдельные
- Взяли из RankMixer token mixing
- История пользователя используется как keys & values в кросс-аттеншене
 - Queries – признаки

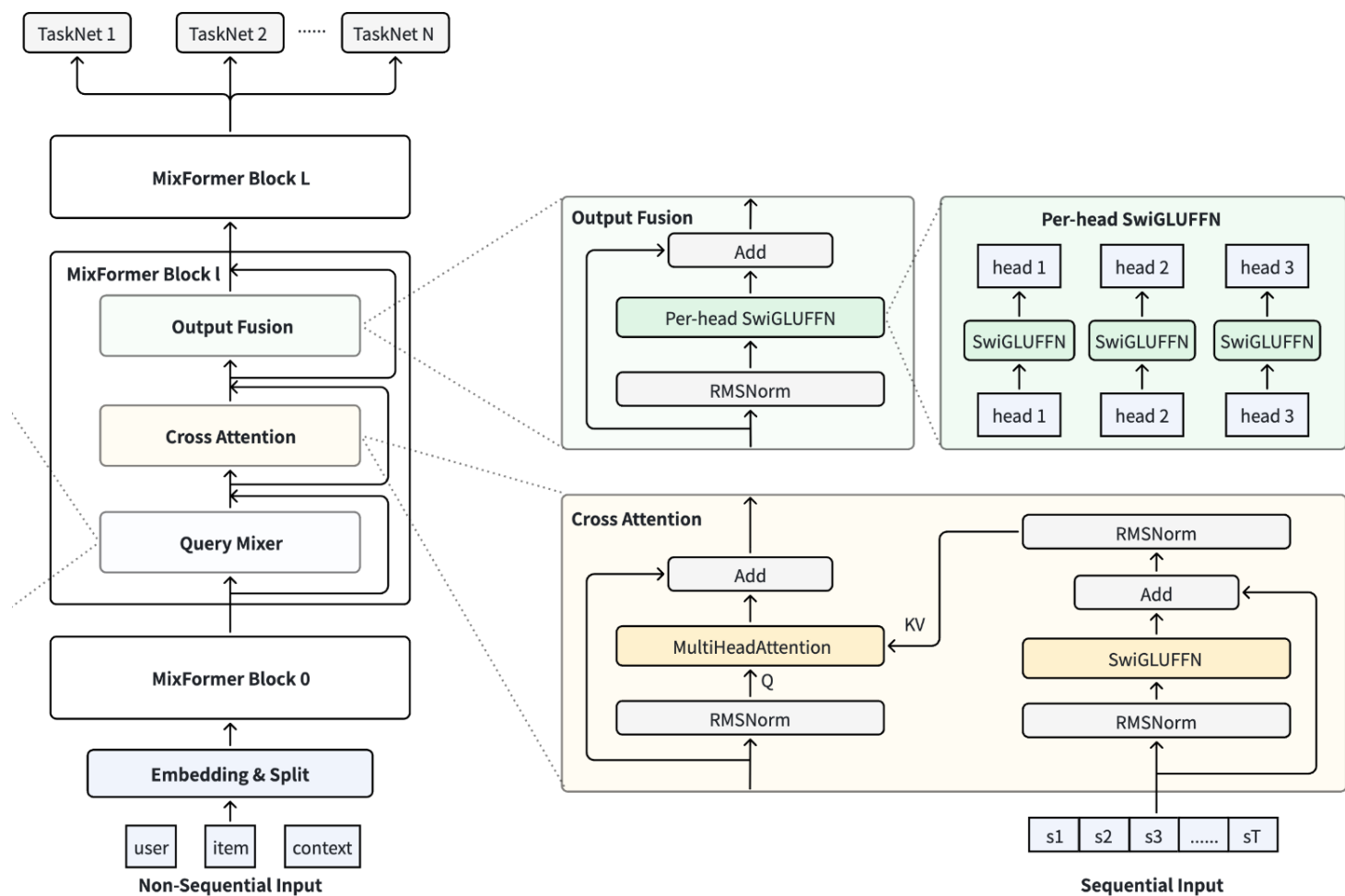


Query mixer

Заменили attention на token mixing (как в RankMixer):



MixFormer



MixFormer

Type	Model	Finish		Skip		Efficiency	
		AUC $\uparrow$	UAUC $\uparrow$	AUC $\uparrow$	UAUC $\uparrow$	#Params(M)	GFLOPs/Batch
Stacked	TA $\rightarrow$ DLRM	0.8554	0.8270	0.8124	0.7294	9	52
	TA $\rightarrow$ DCNv2	+0.13%	+0.13%	+0.15%	+0.26%	22	170
	TA $\rightarrow$ DHEN	+0.18%	+0.26%	+0.36%	+0.52%	22	158
	TA $\rightarrow$ Wukong	+0.29%	+0.29%	+0.49%	+0.65%	122	442
	STCA $\rightarrow$ DCNv2	+0.89%	+0.91%	+1.05%	+1.65%	145	4,560
	TA $\rightarrow$ RankMixer	+0.95%	+1.22%	+1.25%	+1.82%	1,118	2,180
	STCA $\rightarrow$ RankMixer	<u>+1.12%</u>	<u>+1.40%</u>	<u>+1.43%</u>	<u>+2.14%</u>	1,255	6,736
Parallel	OneTrans	+1.05%	+1.31%	+1.30%	+1.95%	316	23,371
	STCA $\oplus$ RankMixer	+1.11%	+1.38%	+1.42%	+2.11%	1,255	6,736
Unified	MixFormer-small	+1.01%	–	+1.18%	–	282	733
	MixFormer-medium	+1.28%	+1.60%	+1.60%	+2.46%	1,226	3,503
	UI-MixFormer-medium	+1.28%	+1.60%	+1.60%	+2.46%	1,226	2,242

MixFormer > OneTrans.

[MixFormer: Co-Scaling Up Dense and Sequence in Industrial Recommenders](#)

MixFormer

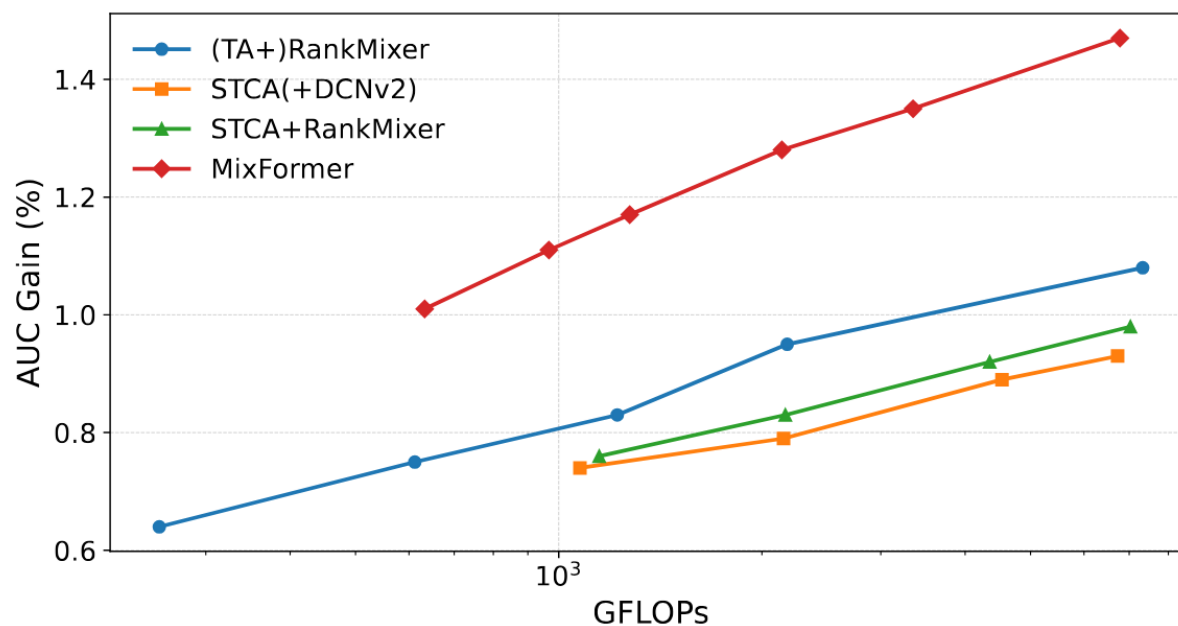


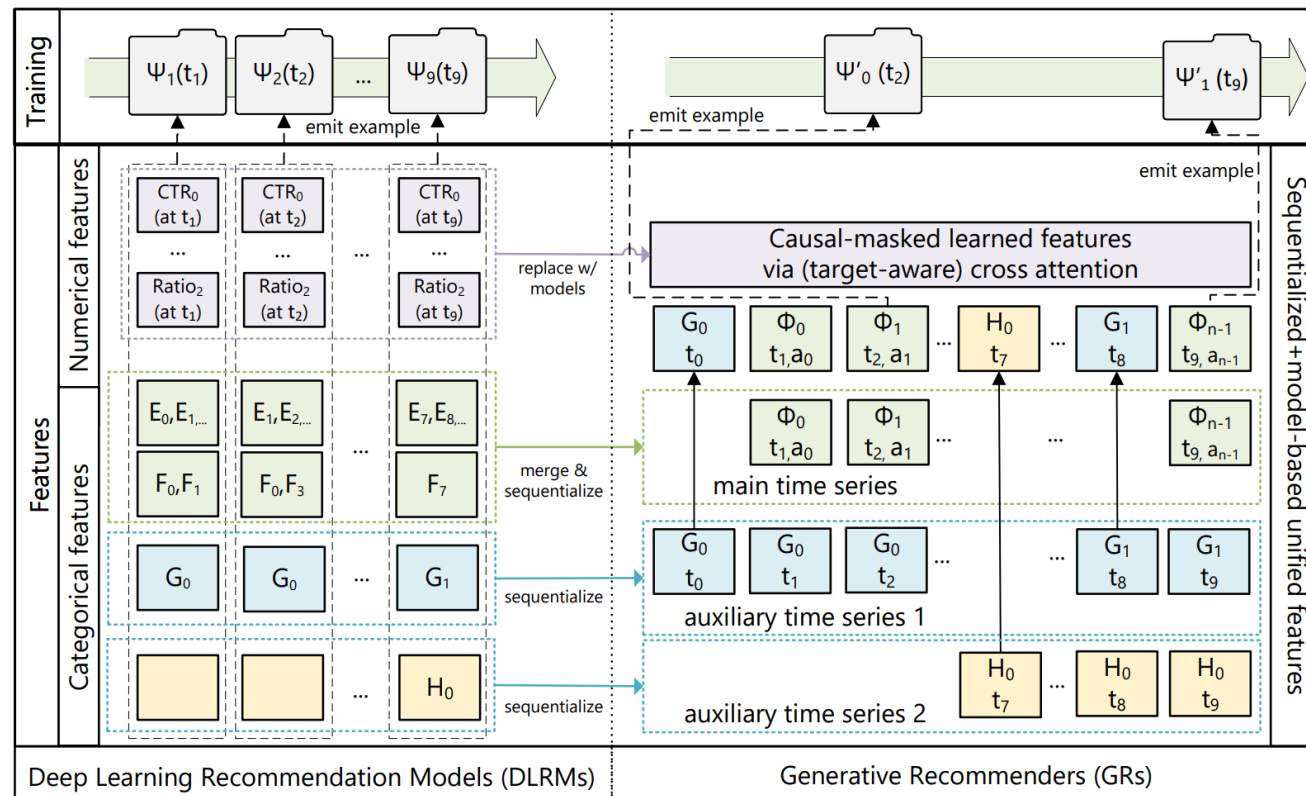
Figure 4: Scaling study over FLOPs. The sequence length is fixed at 512. $(A+)B$ denotes the size of module A is fixed while the size of B is scaled.

МixFormer скейлится по размеру модели.

[MixFormer: Co-Scaling Up Dense and Sequence in Industrial Recommenders](#)

Generative recommenders

- Можно переформулировать задачу ранжирования как генеративную
 - Item, action – токены
 - Предсказываем следующий action токен
- Будем обсуждать дальше в курсе



[Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations](#)

Выводы

- Сначала моделировали пользователя как сумму эмбеддингов прошлых взаимодействий
- Потом перешли на target-aware attention (DIN)
- Потом внедрили полноценный трансформер (BST)
- Текущий тренд – единая модель над user, context, item признаками и историей пользователя

... И всё это делается в supervised парадигме.

Трансформеры для генерации кандидатов и representation learning

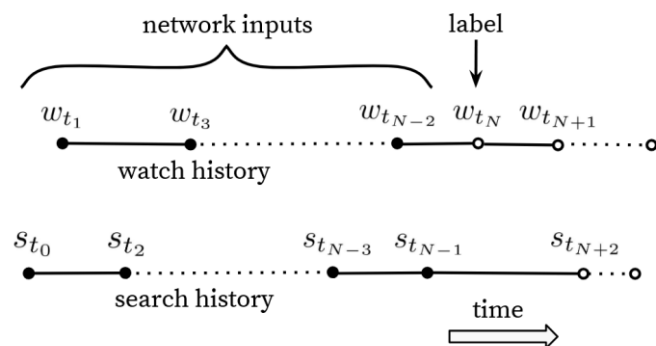
Часть 2

Другие способы обучения и применения трансформеров

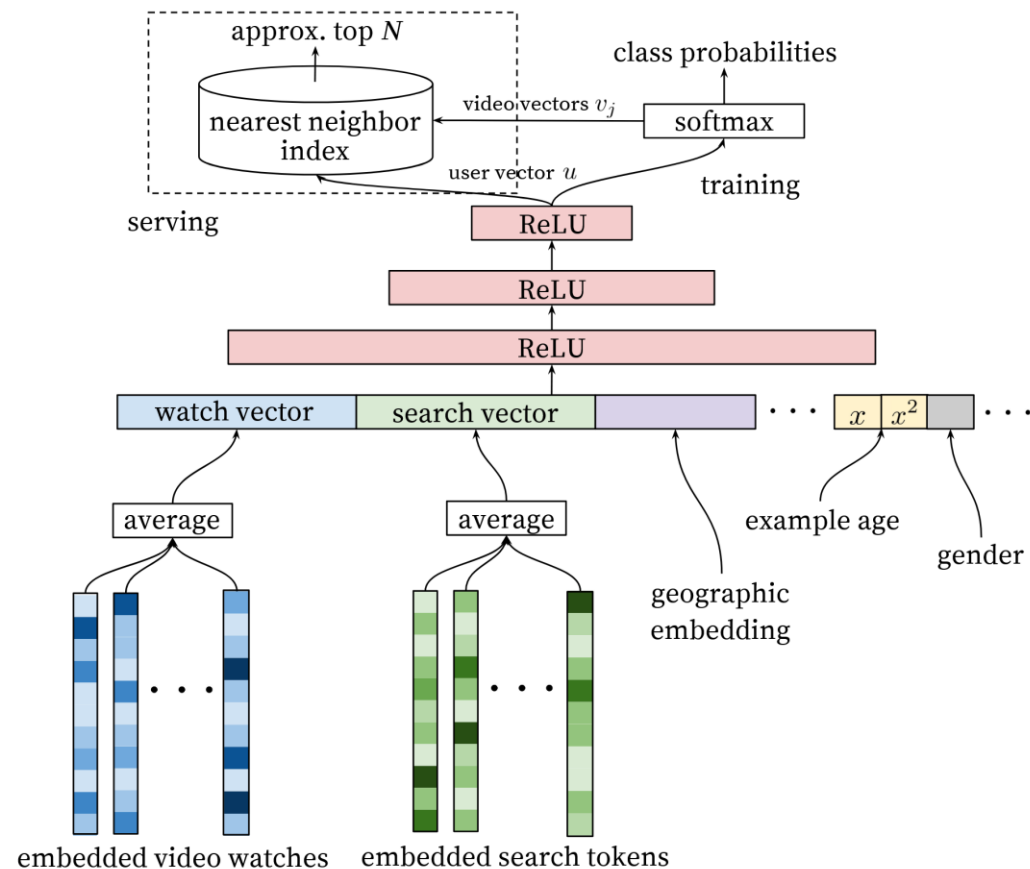
- Для генерации кандидатов нужно учить трансформеры по-другому
 - А именно – на задачу обучения двухбашенных нейросетей для генерации кандидатов
- В ранжировании можно использовать векторы пользователей, которые учились на `unsupervised` задачи

Recap: YouTubeDNN

- Двухбашенная нейросеть
- На входе в башню пользователя – average pooling векторы по истории и признаки
- Задача обучения: предсказание следующего просмотра
- Можно аналогично ранжированию вместо average pooling применить трансформер

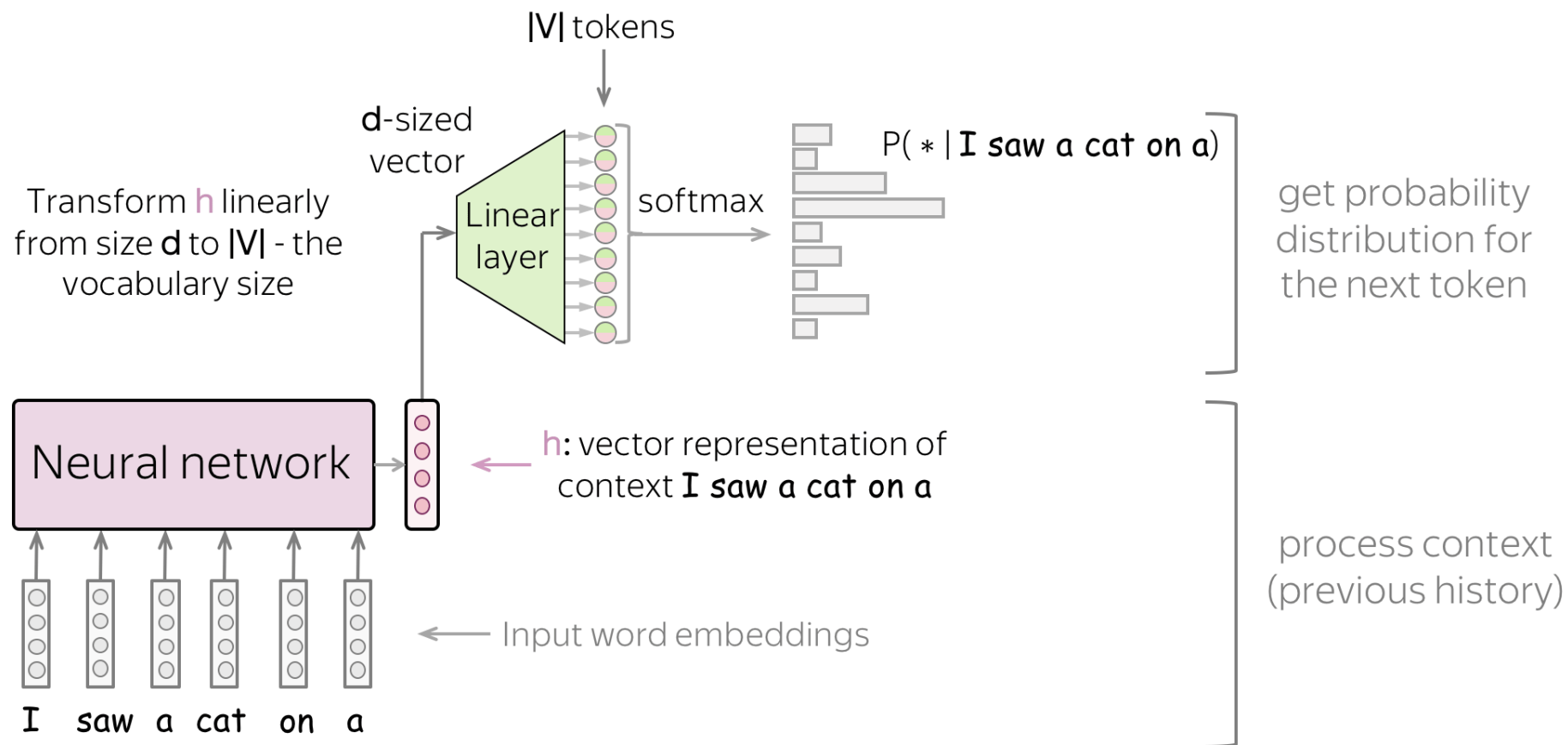


(b) Predicting future watch



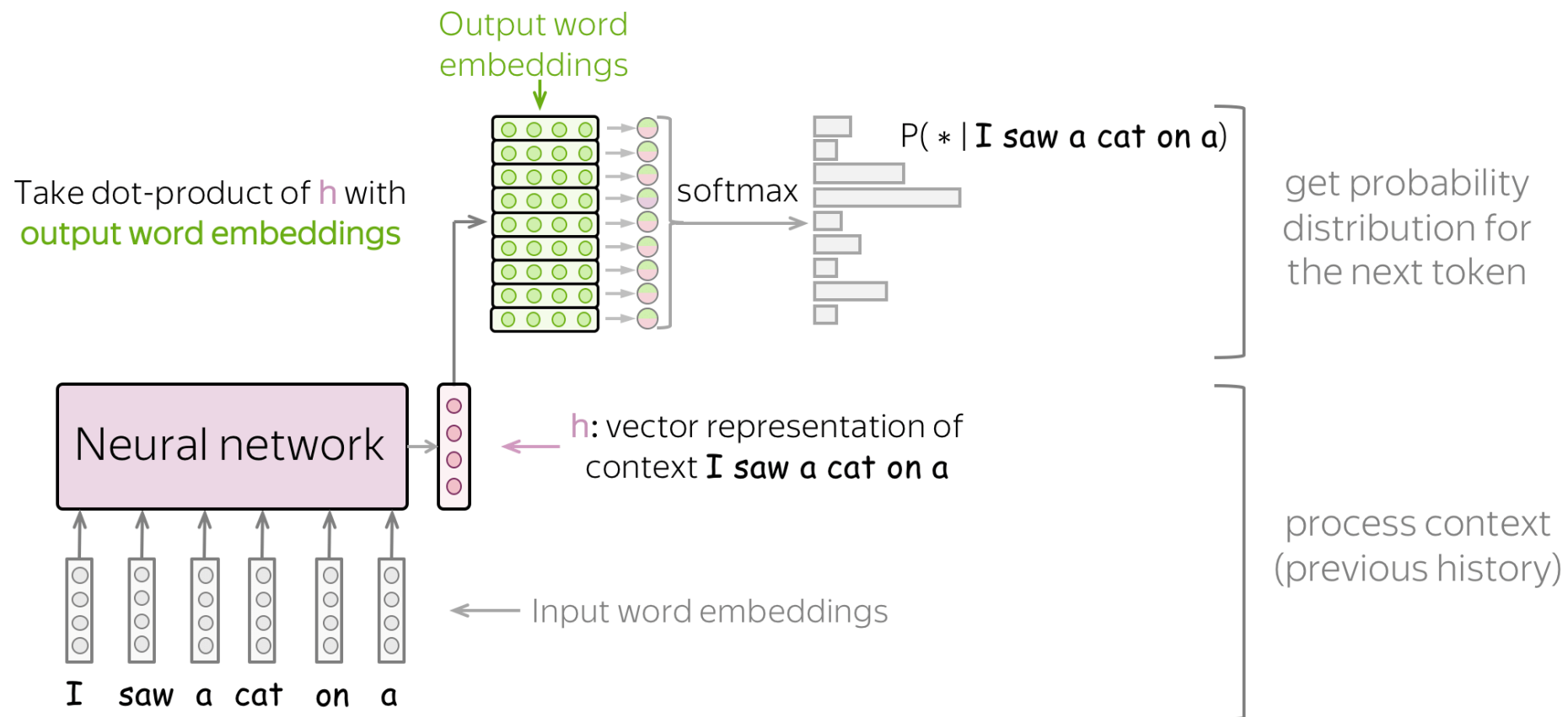
Языковое моделирование

В NLP учим языковые модели предсказывать следующий токен $P(w_t | w_1, \dots, w_{t-1})$:



Языковое моделирование

Это двухбашенная модель с обучаемыми эмбедингами слов:



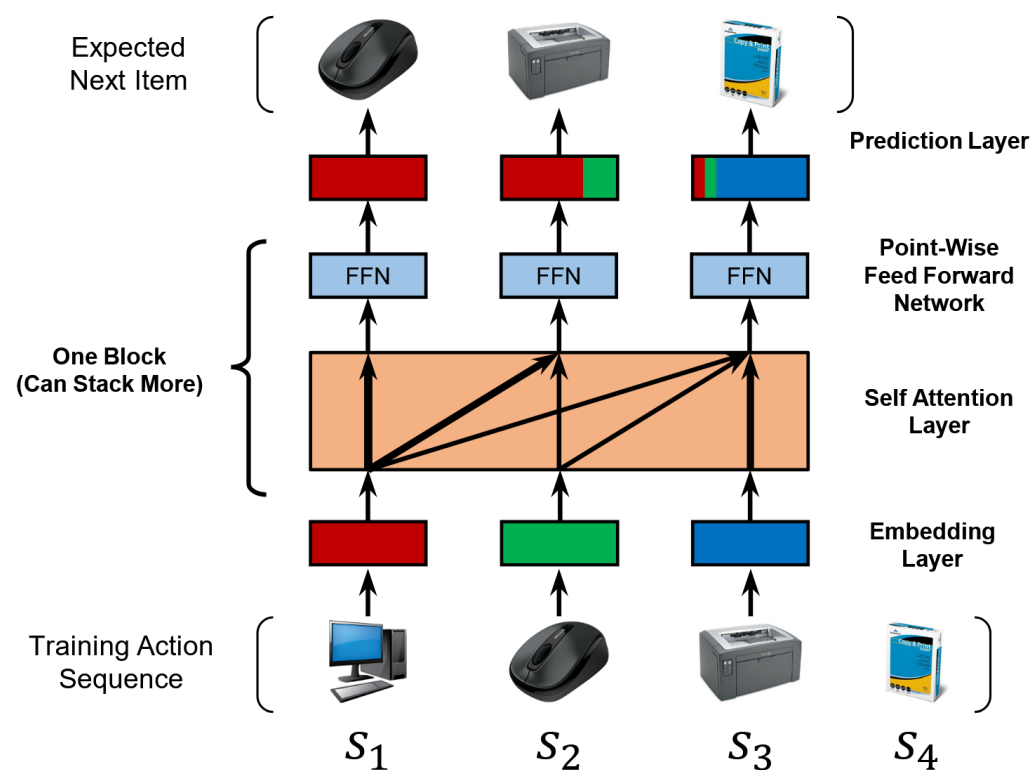
SASRec

Из NLP в RecSys:

- Слова = айтемы, предложения = пользователи
- Next item prediction: предсказываем следующее взаимодействие пользователя
- Трансформер-декодер (с каузальной маской)

До SASRec были аналогичные модели:

- С RNN: [GRU4Rec](#)
- Со свертками: [CASER](#)



SASRec: training objective

- Задача обучения – такая же, как у двухбашенных кандген моделей
 - Учимся на парах (пользователь, позитив)
 - Пользователь – это последовательность прошлых взаимодействий
 - Позитив – следующее взаимодействие
- Но используют более старые и плохие практики чем то, что мы обсуждали на лекциях
 - Binary cross entropy вместо софтмакса
 - Сэмплируют всего один негатив, и тот – равномерно

4Rec

GRU4Rec, SASRec, BERT4Rec, TiSASRec, CL4SRec, CoSeRec, DuoRec, TiCoSeRec, SSE-PT, SSE-SASRec, CARCA, LRURec, S3-Rec, CSRec, gSASRec, UniSRec, MM-Rec, ZESRec, RecGURU, UniM2Rec, MIND, ComiRec, UFORec, HyperRec, Diff4Rec, Caser, STRec, Dcl4Rec, MISSRec, SAMRec, AutoSeqRec, CT4Rec, FMLP-Rec, ConvFormer, GPTRec, LLM4Rec, LLMRec, CFSeRec, FLASH4Rec, AdaptiveRec, NLSIC-BERT4Rec, EASRec, GPTRec, CALRec, SeqRec, ContraRec, AttRec, MOJITO, DiffuRec, STOSA, MoCo4SRec, AutoMLP, UPRec, FNet4Rec, DraG4Rec

На момент 16 ноября 2023 года.

BERT4Rec

- Применили BERT-подобное обучение в рекомендательных системах
 - Вместо каузального – двунаправленный аттеншн
- Чтобы учиться на такую же задачу, как SASRec – нужно отказаться от teacher forcing
 - Не хотим, потому что это очень медленно

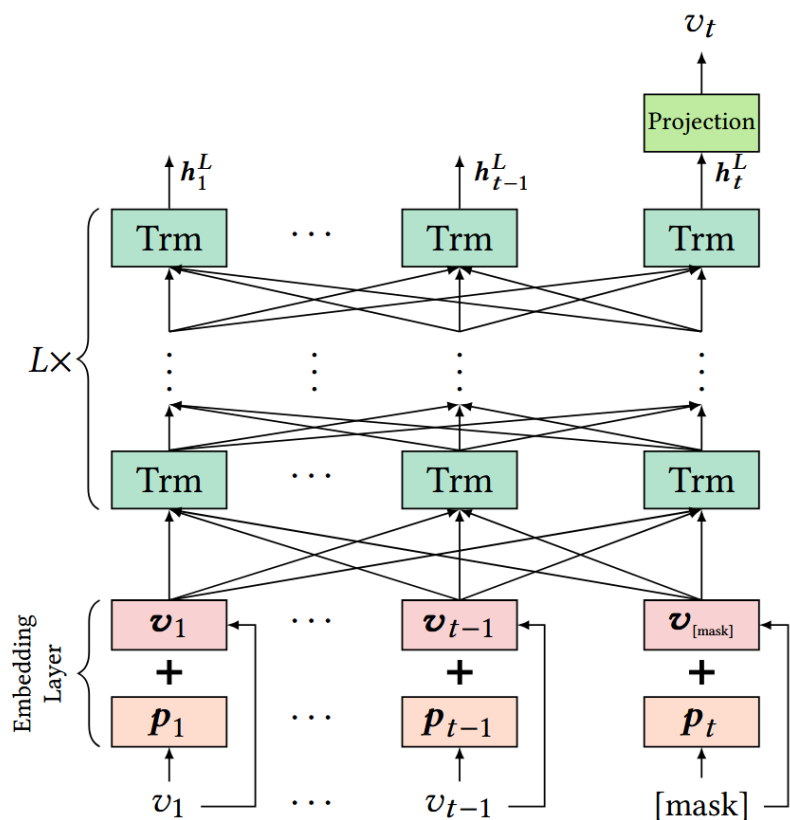
Cloze task

- При обучении маскируем случайные токены в предложении и учим модель их восстанавливать

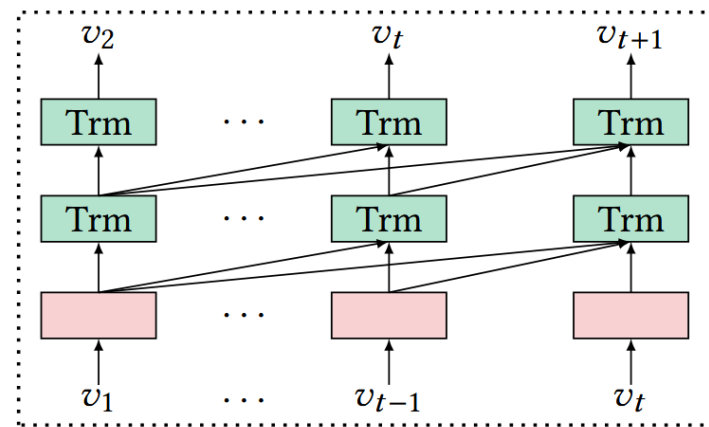
Input: $[v_1, v_2, v_3, v_4, v_5]$ $\xrightarrow{\text{randomly mask}}$ $[v_1, [\text{mask}]_1, v_3, [\text{mask}]_2, v_5]$
Labels: $[\text{mask}]_1 = v_2, \quad [\text{mask}]_2 = v_4$

- При применении добавляем $[\text{mask}]$ в конец последовательности

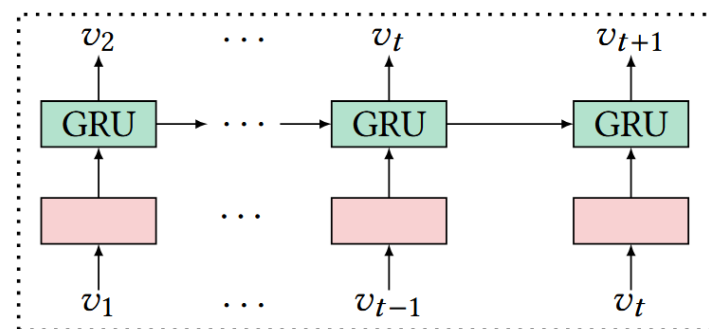
BERT4Rec



(b) BERT4Rec model architecture.



(c) SASRec model architecture.



(d) RNN based sequential recommendation methods.

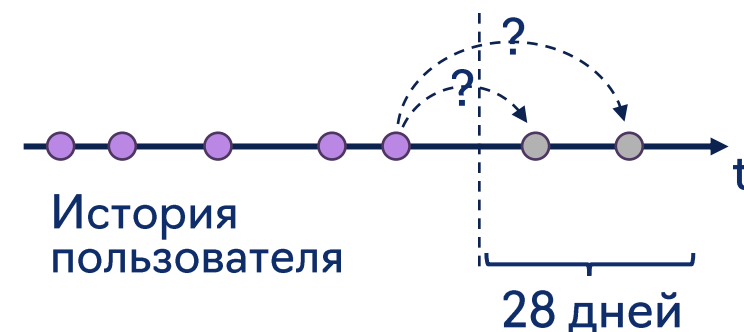
[BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer](#)

BERT4Rec

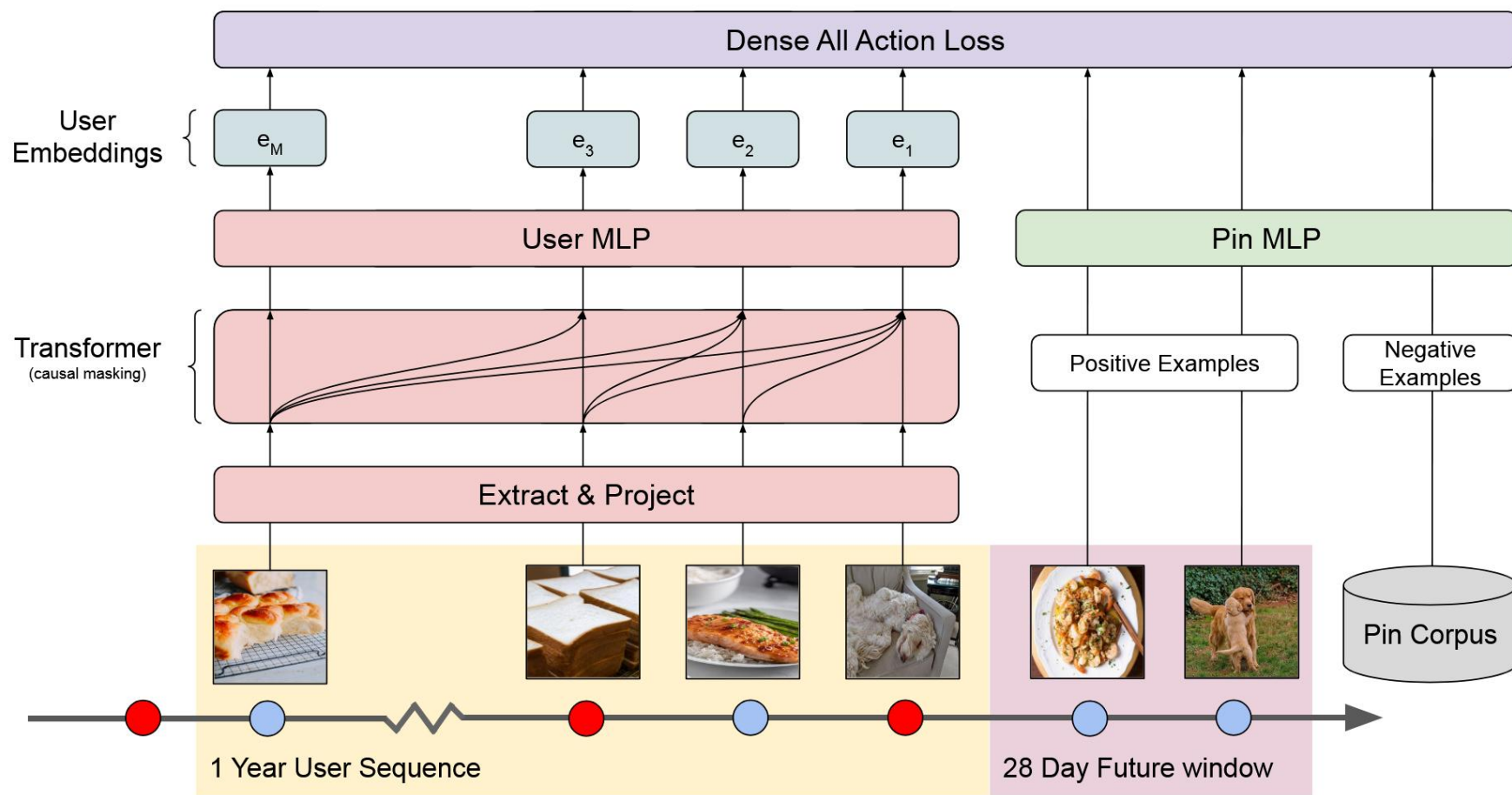
- Показали более хорошие результаты на задаче next-item prediction, чем SASRec
- Учились на полный софтмакс без сэмплирования негативов
- Использовали GELU вместо RELU
- Для оценки качества сэмплировали негативы из униграммного распределения

PinnerFormer

- Первая трансформерная модель над пользователями в Pinterest
- Ежедневно пересчитываются эмбединги на основе последних 256 действий пользователя
- Задача обучения – предсказание будущих положительных взаимодействий в окне 28 дней
- Используют этот вектор на входе в ранжирующую модель
 - Возможно раньше использовали для генерации кандидатов



PinnerFormer



PinnerFormer

Table 2: Real-time vs. offline batch inference. Moving from realtime to batch inference drops Recall@10 by 13.9% when training on SASRec objective, but only by 8.3% when using a dense all action objective (PINNFORMER)

Inference Frequency	Model	R@10	P90 Coverage@10
Once	SASRec	0.198	0.048
	PINNFORMER	0.229	0.042
Daily	SASRec	0.216	0.052
	PINNFORMER	0.243	0.043
Realtime	SASRec	0.251	0.057
	PINNFORMER	0.264	0.045

PinnerFormer лучше SASRec.

[PinnerFormer: Sequence Modeling for User Representation at Pinterest](#)

SASRec++

Table 10: SASRec, Sampled Softmax vs Binary Cross Entropy loss. Sampled softmax performs significantly better than binary cross-entropy on our dataset.

SASRec Loss	Recall@10	P90 Coverage@10
Binary Cross-entropy	0.138	0.111
Sampled Softmax	0.181	0.056
Sampled Softmax + equal e_1 loss weight	0.198	0.048

SASRec с правильным лоссом
гораздо лучше, чем ванильный.

Dataset	Model	HR@10	HR@100	NDCG@10	NDCG@100	Training time	Best epoch
ML-1M	BPR-MF	0.0762	0.3656	0.0383	0.0936	1	60
	GRU4Rec (our)	0.2811	0.6359	0.1648	0.2367	641	90
	BERT4Rec	0.2843	0.6680	0.1537	0.2322	1409	197
	SASRec	0.2500	0.6492	0.1341	0.2153	486	53
	SASRec+ (our)	<u>0.3152</u>	<u>0.6743</u>	<u>0.1821</u>	<u>0.2555</u>	540	63
	SASRec+ 3000 (our)	0.3159	0.6808	0.1857	0.2603	769	85
ML-20M	BPR-MF	0.0806	0.3373	0.0394	0.0892	176	350
	GRU4Rec (our)	0.2813	0.6153	0.1730	0.2401	6319	30
	BERT4Rec	0.2816	0.6311	0.1703	0.2408	14758	68
	SASRec	0.2001	0.5932	0.1067	0.1851	2495	30
	SASRec+ (our)	<u>0.2983</u>	<u>0.6397</u>	<u>0.1833</u>	<u>0.2521</u>	9959	46
	SASRec+ 3000 (our)	0.3090	0.6592	0.1872	0.2581	4125	39
Steam	BPR-MF	0.0431	0.1767	0.0223	0.0480	10	370
	GRU4Rec (our)	0.1138	0.3842	0.0610	0.1138	869	16
	BERT4Rec	0.1242	0.4132	0.0662	0.1228	4893	74
	SASRec	0.0981	0.3608	0.0506	0.1016	2140	39
	SASRec+ (our)	0.1191	0.3947	0.0641	0.1179	1262	16
	SASRec+ 3000 (our)	<u>0.1206</u>	<u>0.3974</u>	<u>0.0652</u>	<u>0.1192</u>	1226	14
Beauty	BPR-MF	0.0271	0.0970	0.0144	0.0280	1	400
	GRU4Rec (our)	0.0291	0.0933	0.0163	0.0288	644	25
	BERT4Rec	0.0338	0.1051	0.0187	0.0325	2325	87
	SASRec	0.0246	0.0939	0.0126	0.0262	521	26
	SASRec+ (our)	0.0533	0.1325	0.0327	0.0482	332	6
	SASRec+ 3000 (our)	<u>0.0490</u>	<u>0.1197</u>	<u>0.0295</u>	<u>0.0434</u>	296	8
Yelp	BPR-MF	0.0176	0.0967	0.0087	0.0236	1	100
	GRU4Rec (our)	0.0425	0.1822	0.0216	0.0483	2677	5
	BERT4Rec	0.0442	0.1912	0.0223	0.0505	10166	21
	SASRec	0.0228	0.1067	0.0115	0.0274	655	3
	SASRec+ (our)	0.0482	0.2005	0.0246	0.0539	2505	3
	SASRec+ 3000 (our)	<u>0.0462</u>	<u>0.1929</u>	<u>0.0237</u>	<u>0.0519</u>	1965	5

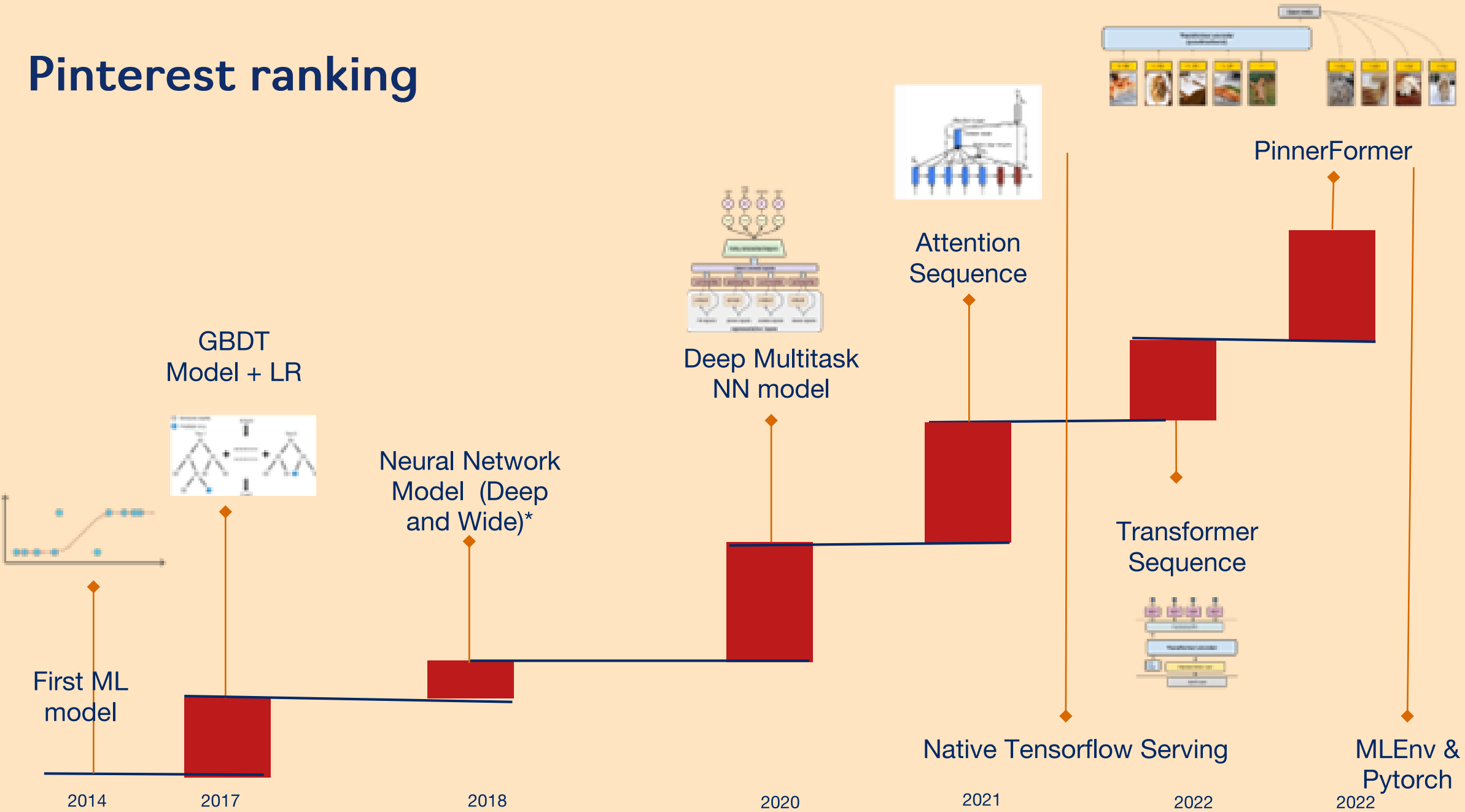
SASRec++ > BERT4Rec.

[Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec?](#)

Выводы

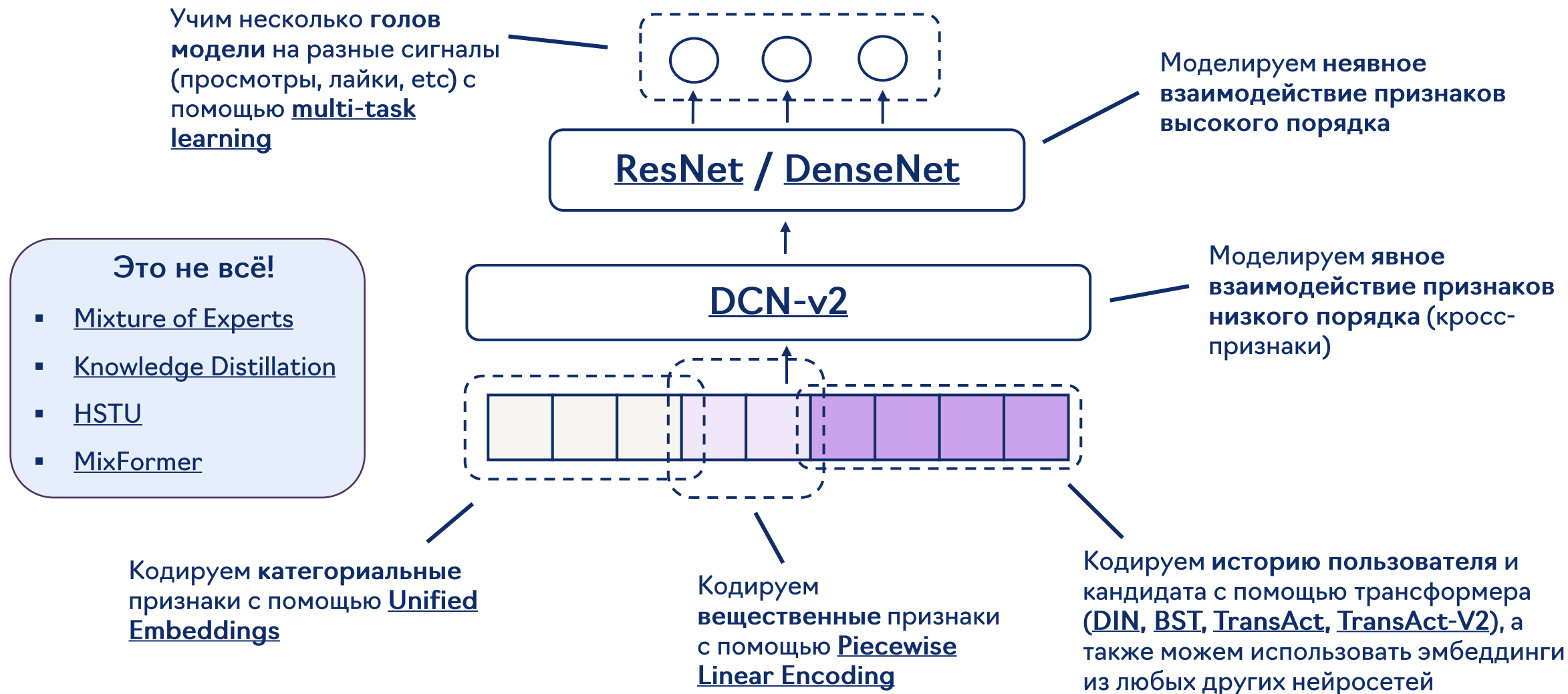
- Можно учить двухбашенные кандген модели с трансформером в качестве энкодера пользователя
- Можем эффективно учить такие модели с teacher forcing как в языковых моделях
- Можно также использовать такие модели как representation learning (переиспользовать векторы для ранжирования)

Pinterest ranking



Evolution of Ads Ranking at Pinterest by Aayush Mudgal

Нейросетевое ранжирование

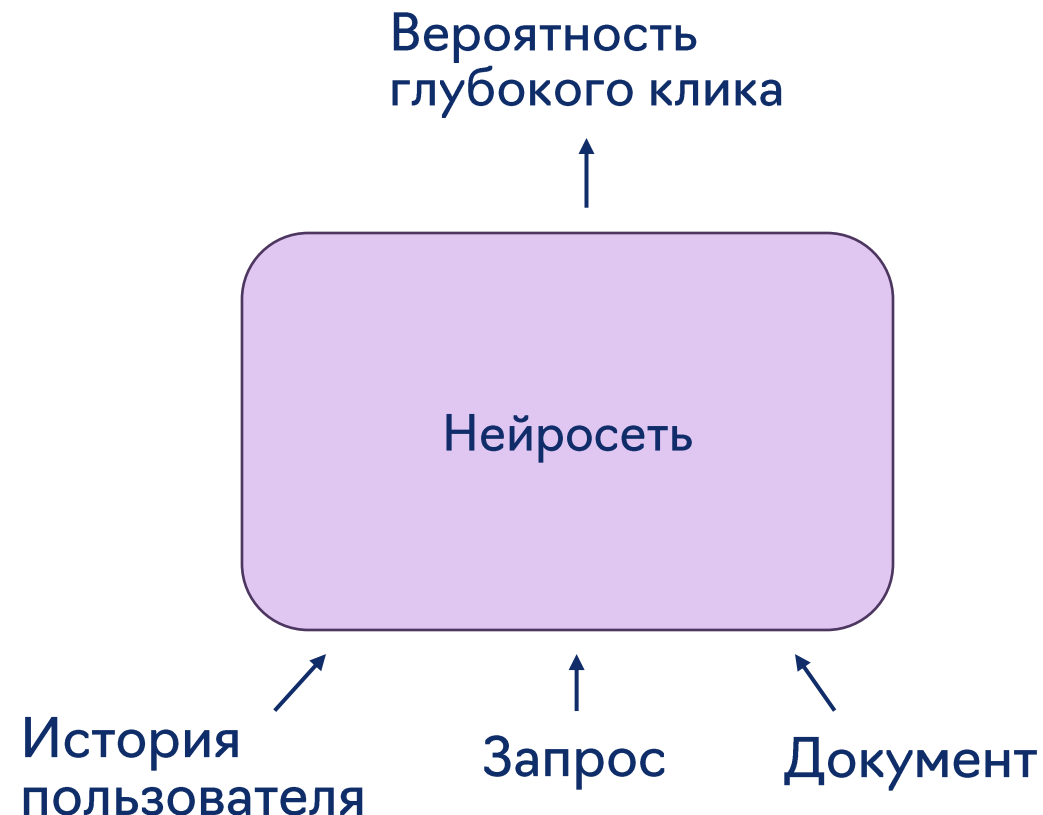


Рекомендательные трансформеры в Яндексе

Часть 3

Первый трансформер: веб-поиск

- Задача – персонализация поиска
- На входе в модель история поисковых запросов и кликов на документы
 - Именно кликов, не показов. Почему?
- Предсказываем вероятность глубокого клика (время пребывания на документе больше N секунд)
 - Как посчитать время пребывания?

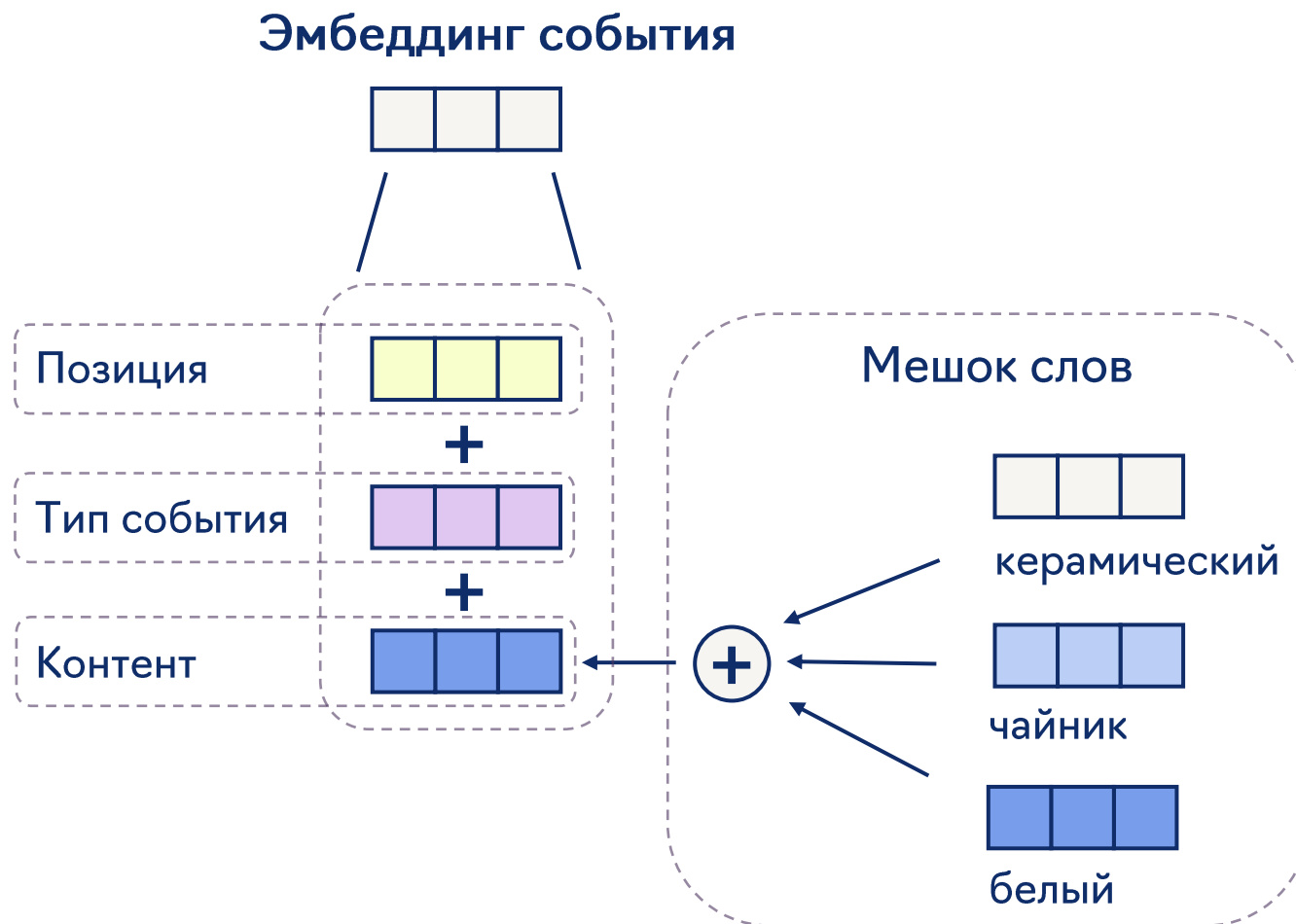


Первый трансформер: кодирование событий

Эмбединг события – сумма контента, позиции и типа события.

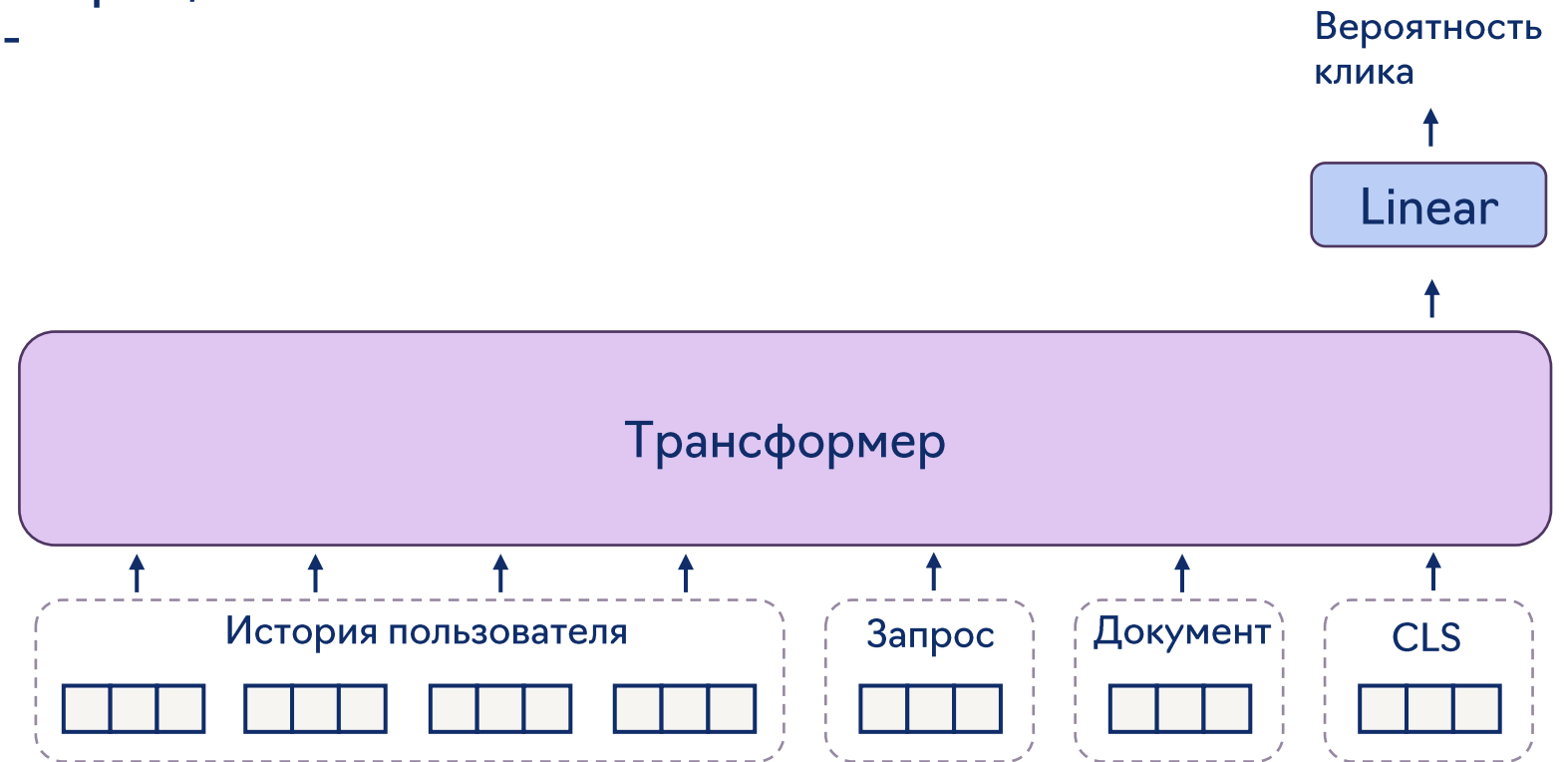
Токенизация:

- Начинали с DSSM-подобной схемы с униграммами, биграммами, чарттриграммами
- Перешли на BPE-токенизацию с domain-specific словарём (а потом и на sentencepiece)



Первый трансформер: раннее связывание

- Раннее связывание истории, запроса и документа-кандидата
- Двухнаправленный трансформер

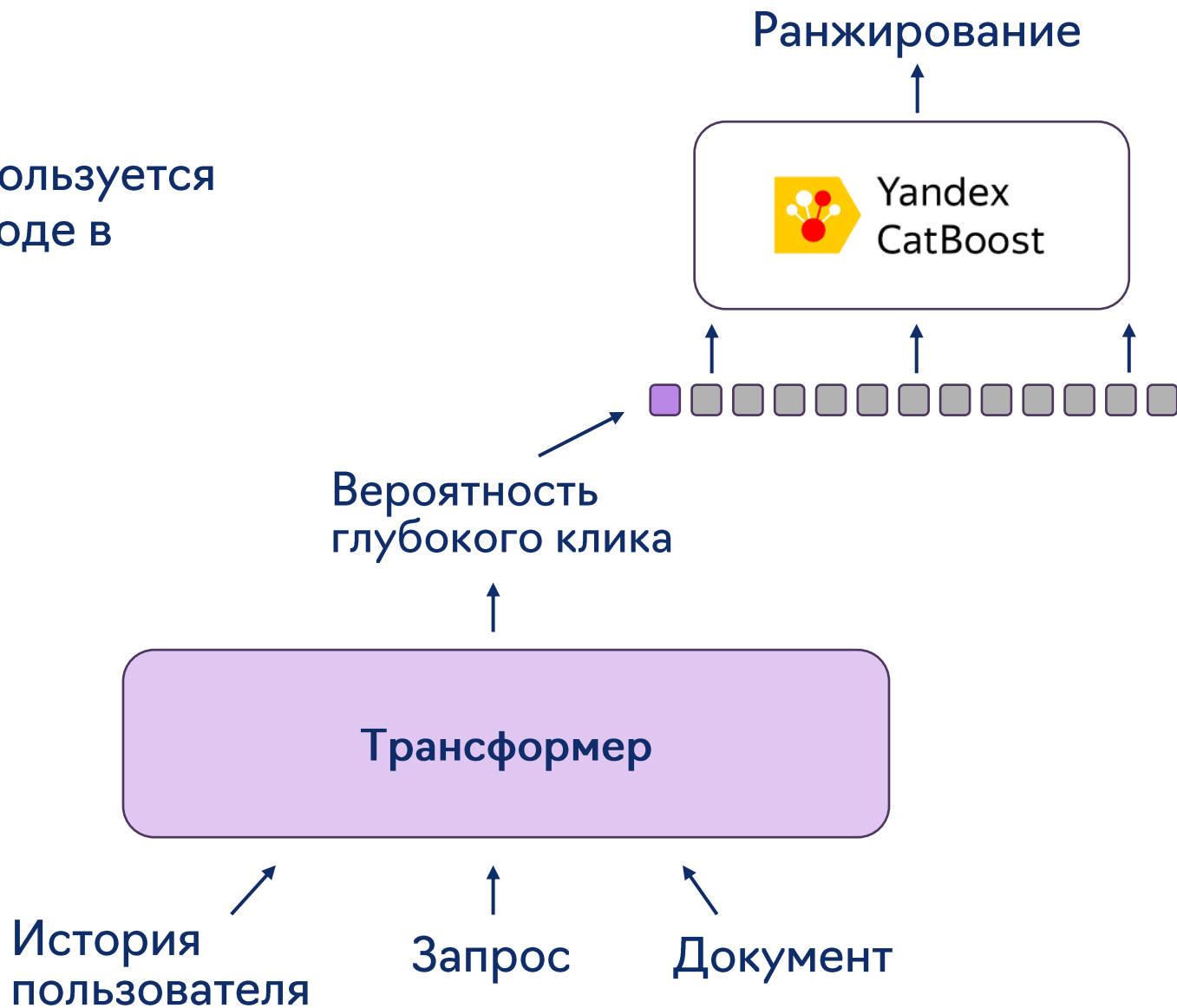


Первый трансформер: обучение

- Для каждого пользователя за большой промежуток времени (год?) формируем обучающие примеры:
 - Сэмпл – тройка (история, запрос, показанный документ)
 - Таргет – был глубокий клик или нет
 - Функция потерь – бинарная кросс-энтропия
- Было предобучение, похожее на BERT
 - Оставляем показы без кликов в истории, для части документов в истории прячем информацию о клике, восстанавливаем где был клик

Первый трансформер: внедрение

- Предсказание модели используется как мощный признак на входе в ранжирующий катбуст



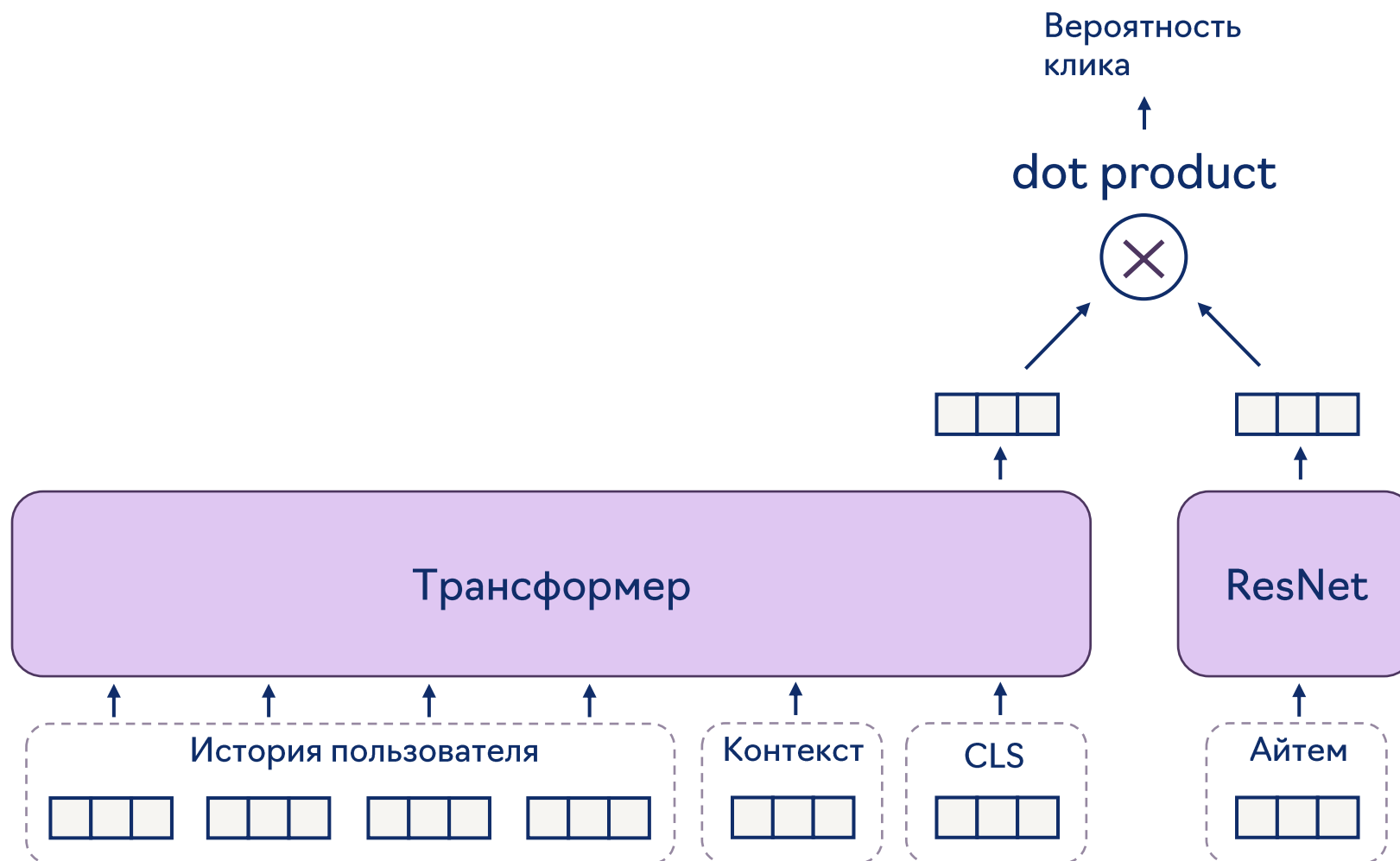
Трансформеры для рекламы

- Вместо запроса – контекст
 - Текущая веб-страница, устройство, операционная система, etc
 - В поисковой рекламе контекст – опять запрос
- История – клики на рекламу и поисковые запросы
- Важны ID-based фичи
 - E.g., Page ID; Impression ID (номер рекламного блока)
 - Обучаемые эмбединги с клиппингом по популярности (без них модель не дает прирост качества)
- Важно регулярное дообучение модели на новых данных

Трансформеры для рекламы

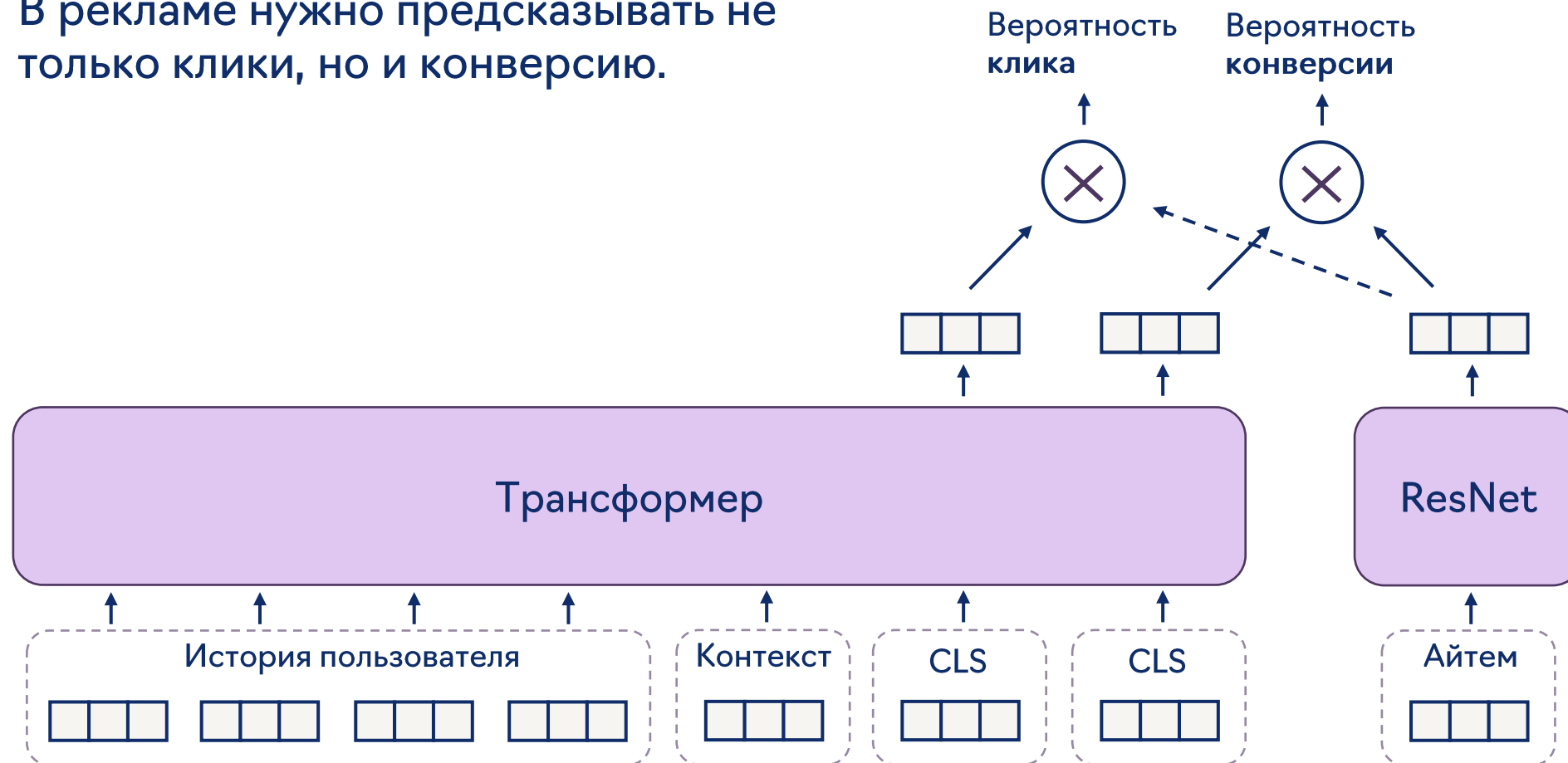
- В рекламе очень высокая нагрузка
- В модели с ранним связыванием нужно на каждого кандидата пересчитывать трансформер над историей – это очень тяжело
- Нужно было сделать позднее связывание вместо раннего

Двухбашенная архитектура



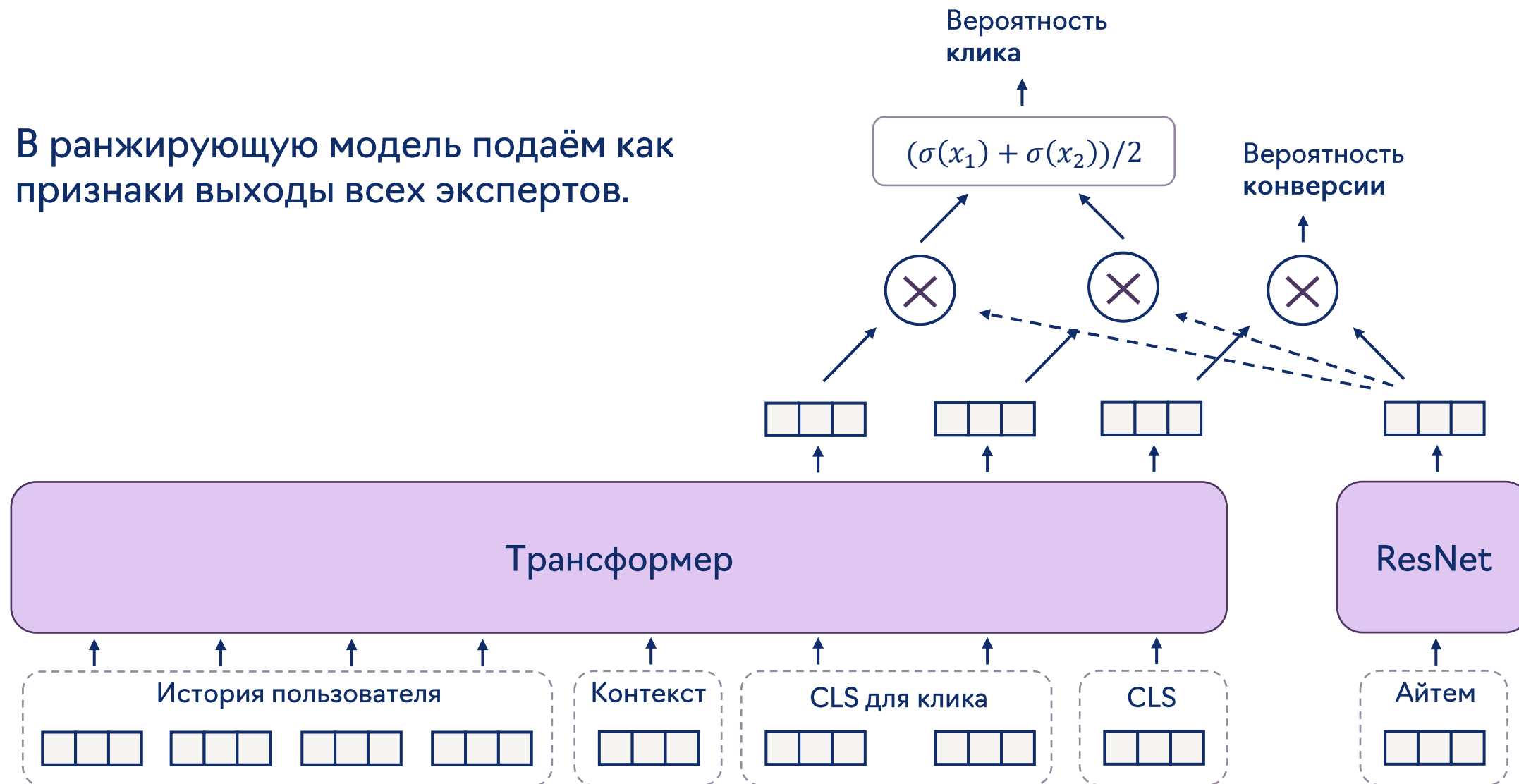
Multi-task learning

В рекламе нужно предсказывать не только клики, но и конверсию.



Смесь экспертов

В ранжирующую модель подаём как признаки выходы всех экспертов.



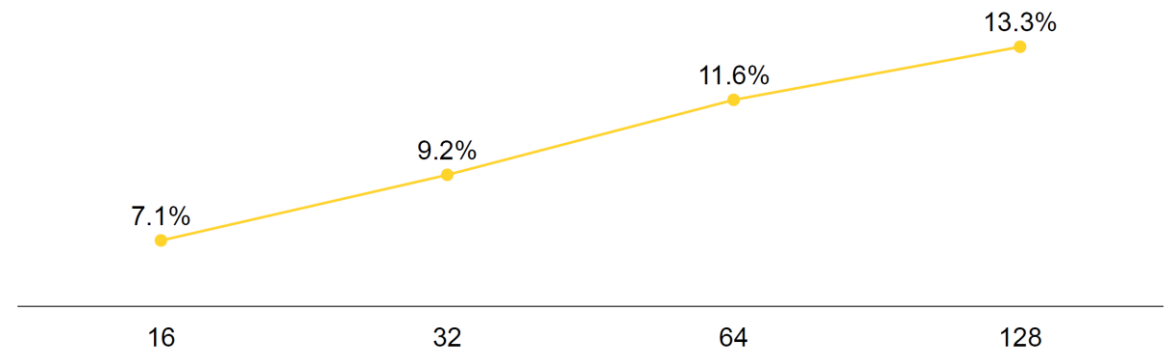
Рекламные трансформеры: обучение

- Объединяем всех кандидатов из одного реквеста (из одной выдачи) в единый сэмпл
- Учим не только pointwise, но pairwise / listwise задачи (ранжировать айтемы из одной выдачи)
- Добавили предобучение на contrastive learning на парах (пользователь, его клик)
- В более поздних итерациях добавили дистилляцию из большого учителя

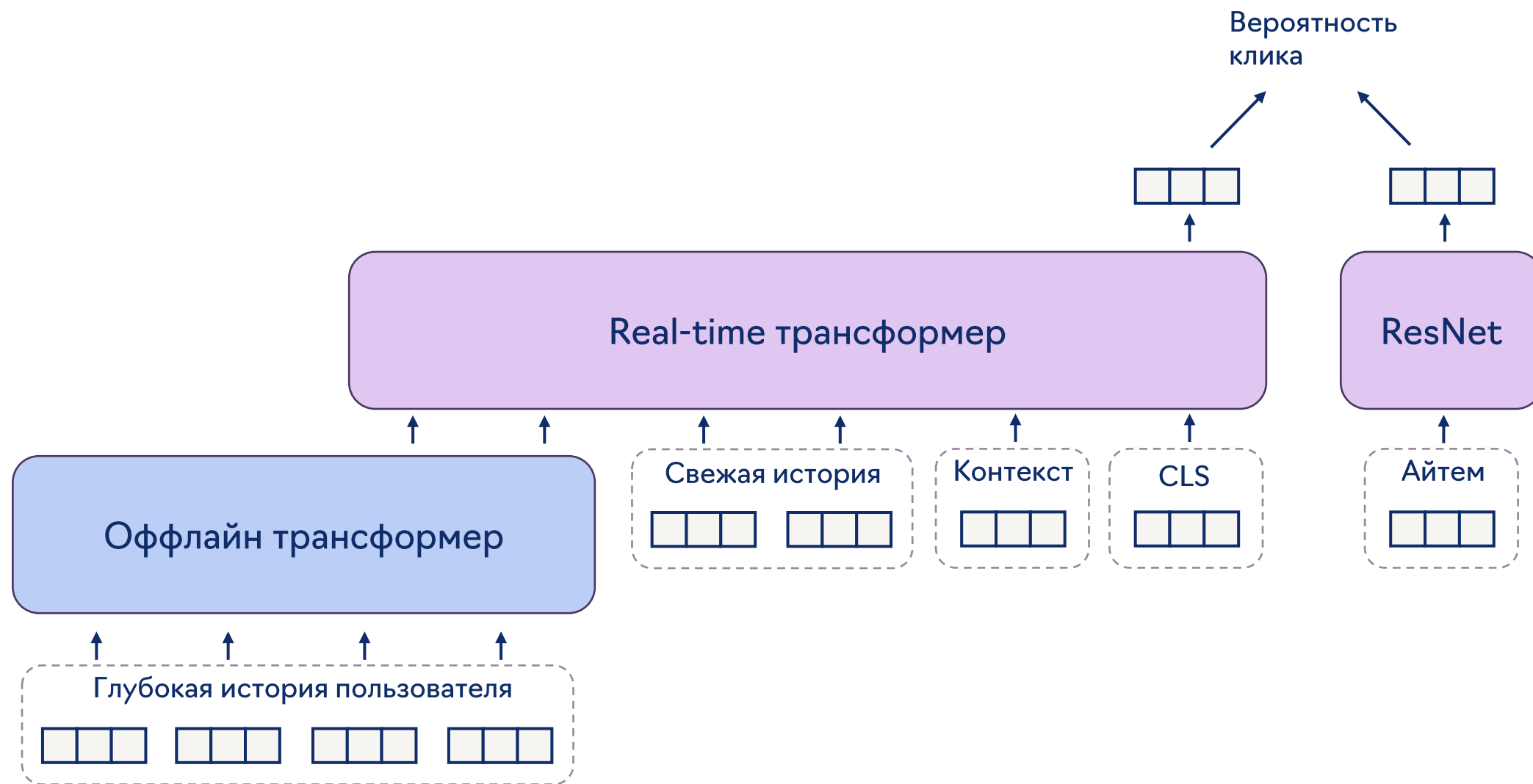
Длина истории

- Хотим анализировать глубокую историю пользователя
- Просто добавить её в real-time трансформер нельзя – это сильно замедлит рекомендации
- Что можно сделать:
 - Event picking – брать не последние n событий, а выбирать по какому-то более хитрому принципу среди глубокой истории
 - Offline модель

Профит от увеличения длины истории

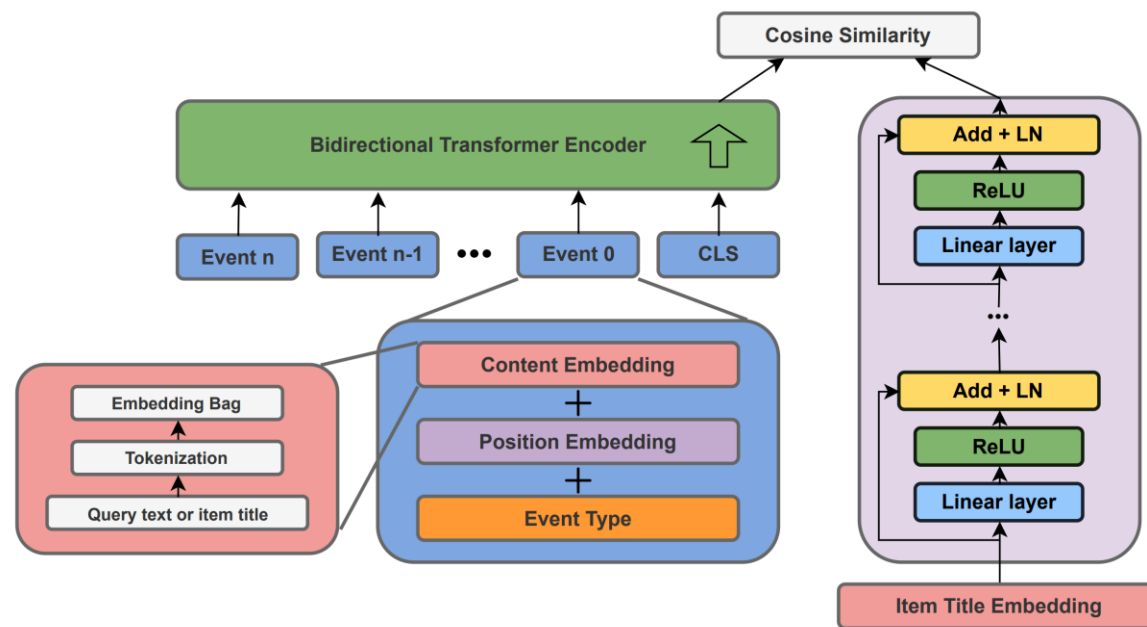


Offline-realtime модель



Оффлайн модели

- Раз в день пересчитываем эмбединги пользователей и айтемов
- Расчет делается в оффлайн контуре
 - Гораздо проще разрабатывать и внедрять модели
 - Можно внедрять более тяжелые модели за те же ресурсы
 - Смогли сделать длину истории до двух тысяч и в 2-3 раза увеличить модель
- Минусы:
 - Не учитываем свежую историю
 - Не учитываем контекст



Генерация кандидатов

- Начали делать кандген трансформеры позже ранжирующих
- Учим модель предсказывать айтем, с которым будет положительное взаимодействие в будущем
 - К позитивам приклеиваем историю с лагом в один день
- Sampled softmax loss с logQ correction
- Начали делать многоголовые модели сразу на ранжирование и кандген

Мульти-доменные модели

- Кросс-доменные модели: история из одного сервиса переиспользуется для рекомендаций в другом
 - E.g. веб-запросы в рекомендациях Маркета
- Сделали объединенную модель для поиска и рекомендаций Маркета:
 - Единый каталог айтемов
 - Объединенная история пользователя
 - Различается только контекст

История пользователя	Прирост в ранжировании от трансформера
Яндекс Маркет	+1.645%
С веб-запросами	+2.061%

Что потом

- Перешли к авторегрессивному обучению и замасштабировали модели
 - Длина истории до 8к событий
 - Количество параметров 10x
 - Про это позже в курсе
- Начали использовать **unified embeddings** (вместо обучаемых эмбеддингов с клиппингом по популярности)

Кодирование айтема	Прирост в ранжировании от трансформера
Только контент	+0.506%
Только айдишник	+0.767%
Контент + айдишник	+1.250%

Выводы

- Ранжирующий трансформер с ранним связыванием для веб-поиска
- Рекламные трансформеры
 - Двухбашенные модели
 - Многоголовые – клики и конверсии, а также смесь экспертов на одну задачу
 - Оффлайн-рилтайм архитектура
- Оффлайн модели (для Музыки и Маркета)
 - Большая длина истории и много параметров
- Трансформеры для генерации кандидатов
- Мульти-доменные модели для поиска и рекомендаций

Выводы

Обсудили:

- Трансформерные над историей пользователя для ранжирования
- Next item prediction модели и representation learning

На семинаре – разбор статей про модели от Pinterest – [PinnerFormer](#) и [TransAct](#).